COMSATS University Islamabad (CUI)

Software Requirement Specification
(SRS DOCUMENT)

for

PodVerse

Version 1.0

By

Rabeea Chughtai **CIIT/SP22-BCS-079/ISB**
Sadia Imran **CIIT/SP22-BCS-087/ISB**

Supervisor
Ms. Saadia Maqbool

Bachelor of Science in Computer Science (20xx-20xx)

Revision History

Name	Date	Reason for Changes	Version

Application Evaluation History

Comments (by committee) ***include the ones given at scope time both in doc and presentation**	**Action Taken**

Supervised by

Supervisor's Name

Signature_____

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Modules 2
1.3.1 Module 1: User Access & Profile Management 2
1.3.2 Module 2: Voice Cloning 2
1.3.3 Module 3: Knowledge Base Setup 2
1.3.4 Module 4: RAG Retrieval 2
1.3.5 Module 5: Agent Customization 2
1.3.6 Module 6: Podcast Audio Creation 3
1.3.7 Module 7: AI Host 3
1.3.8 Module 8: AI Guest 3
1.3.9 Module 9: Podcast Video Creation & Publishing 3
1.3.10 Module 10: Subscription & Payment 4
1.3.11 Module 11: FAQs Chatbot 4
1.3.12 Module 12: Audience Engagement and Communication 4
1.4 Overview 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 User classes and characteristics 5
2.3 Operating Environment 6
2.4 Design and Implementation Constraints 6
3. Requirement Identifying Technique 6
3.1 Use Case Diagram 6
3.1.1 Creator 7
3.1.2 Viewer 8
3.2 Business Rules 9
3.3 Detail Use Cases 9
3.3.1 Module 1 : User Access & Profile Management 9
3.3.2 Module 2 : Voice Cloning 24
3.3.3 Module 3 : Knowledge Base Setup 25
3.3.4 Module 5 : Agent Customization 27
3.3.5 Module 6 : Podcast Audio Creation 28
3.3.6 Module 7 : AI Host & Module 8 : AI Guest 33
3.3.7 Module 9 : Podcast Video Creation & Publishing 35
3.3.8 Module 10 :Subscription and Payment 42
3.3.9 Module 11: FAQs Chatbot 46
3.3.10 Module 12: Audience Engagement and Communication 49
3.4 Event Response Table 54
3.4.1 ERT-1: User Login 54
3.4.2 ERT-2: Profile Update 54
3.4.3 ERT-3: Voice Cloning 55
3.4.4 ERT-4: Knowledge Base Upload 55
3.4.5 ERT-5: Chatbot Query Processing 55
3.4.6 ERT-6: Terminate Test Section 56
3.4.7 ERT-7: AI Host Podcast Management 56
3.4.8 ERT-8: AI Guest Response 56
3.4.9 ERT-9: Podcast Recording 57
3.4.10 ERT-10: User Edits Podcast Recording 57
3.4.11 ERT-11: User Exports Podcast Audio/Video 57

3.4.12 ERT-12: User Subscribes to Premium Plan 58
3.4.13 ERT-13: Payment Fails 58
3.4.14 ERT-14: User Rates a Podcast Session 59
3.4.15 ERT-15: System Generates Podcast Summary 59
3.4.16 ERT-16: User Interacts with AI-Powered Notifications 59
4. Functional Requirements 60
4.1 UC-1 Register Account 60
4.1.1 FR-1: Account.Register 60
4.1.2 FR-2: Account.Register.SocialLogin 60
4.1.3 FR-3: Account. ValidateEmail 61
4.1.4 FR-4: Account.EmailVerification 61
4.1.5 FR-5: Account.EmailVerification.Expiration 62
4.1.6 FR-6: Account.PasswordStrengthValidation 63
4.2 UC-1.2 User Login 64
4.2.1 FR-7: Account.Login 64
4.2.2 FR-8: Account.Authenticate 64
4.2.3 FR-9: Account.LoginError 65
4.3 UC-1.3 Google Authentication 65
4.3.1 FR-10: Account.GoogleLogin 66
4.3.2 FR-11: Account.GoogleRedirect 66
4.3.3 FR-12: Account.GoogleCredentialVerification 67
4.3.4 FR-13: Account.GoogleAutoRegister 67
4.3.5 FR-14: Account.GoogleLinkedLogin 68
4.3.6 FR-15: Account.GoogleAuthFailure 68
4.4 UC-1.4 Password Recovery 69
4.4.1 FR-16: Account.PasswordResetRequest 69
4.4.2 FR-17: Account.ResetLinkGeneration 69
4.4.3 FR-18: Account.ResetLinkExpiration 70
4.4.4 FR-19: Account.InvalidResetRequest 70
4.4.5 FR-20: Account.NewPasswordValidation 71
4.4.6 FR-21: Account.PasswordUpdate 71
4.5 UC-1.5 Logout 72
4.5.1 FR-22: Account.Logout 72
4.5.2 FR-23: Account.SessionTermination 72
4.5.3 FR-24: Account.LogoutConfirmation 73
4.6 UC-1.6: Access Sample Podcasts 73
4.6.1 FR-25: Podcasts.ViewSamples 73
Title 73
4.6.2 FR-26: Podcasts.SearchSamples 74
Identifier 74
4.7 UC-1.7: Start a Podcast Session 74
4.7.1 FR-27: PodcastSession.Start 74
Identifier 74
4.7.2 FR-28: PodcastSession.Resume 74
4.8 UC-1.8: View Dashboard 75
4.8.1 FR-29: Dashboard.ViewContent 75
Title 75
4.8.2 FR-30: Dashboard.HandleNoConnection 76
Identifier 76
Title 76
Dependencies 76
4.9 UC-1.9: Create and Personalize User Profile 76
4.9.1 FR-31: Profile.Create 76
Title 76

4.9.2 FR-32: Profile.Update 77
Title 77
4.10 UC-1.10: Edit Profile 77
4.10.1 FR-33: Profile.EditDetails 77
4.10.2 FR-34: Profile.ValidatePassword 77
Title 78
4.11 UC-1.11: Follow Creators and View Updates 78
4.11.1 FR-35: FollowCreator 78
Title 78
4.11.2 FR-36: ViewCreatorUpdates 78
Title 79
4.12 UC-1.12: View Activity Feed of Followed Creators 79
4.12.1 FR-37: ActivityFeed.View 79
Title 79
4.12.2 FR-38: ActivityFeed.HandleNoData 79
FR-38 79
Title 80
4.13 UC-2.1 Voice Cloning Setup 80
4.13.1 FR-39: VoiceCloning.RecordSample 80
FR-39 80
Title 80
4.13.2 4.13.2 FR-40: VoiceCloning.UploadSample 80
Identifier 80
4.13.3 FR-41: VoiceCloning.ProcessSample 81
Title 81
4.13.4 FR-42: VoiceCloning.PreviewGeneratedVoice 81
Title 81
4.14 UC-3.1 Knowledge Base Setup 82
4.14.1 FR-43: KnowledgeBase.UploadData 82
4.14.2 FR-44: KnowledgeBase.ProcessData 82
4.14.3 FR-45: KnowledgeBase.VectorEmbedding 83
4.14.4 FR-46: KnowledgeBase.MetadataTagging 83
4.15 UC-4.1 AI Agent Customization 84
4.15.1 FR-47: AIAgent.CustomizeAppearance 84
4.15.2 FR-48: AIAgent.SavePreferences 84
4.15.3 FR-49: AIAgent.LoadPreferences 85
4.16 UC-5.1 Record Podcast Session 85
4.16.1 FR-50: PodcastSession.StartRecording 85
4.16.2 FR-51: PodcastSession.PauseRecording 86
4.16.3 FR-52: PodcastSession.StopRecording 86
4.16.4 FR-53: PodcastSession.SetRecordingLimit 86
4.16.5 FR-54: PodcastSession.AutoSaveRecording 87
4.17 UC-5.2 Edit Podcast Recording 87
4.17.1 FR-56: PodcastSession.LoadEditingInterface 87
4.17.2 FR-57: PodcastSession.TrimAudio 88
4.17.3 FR-58: PodcastSession.RearrangeClips 88
4.17.4 FR-59: PodcastSession.MergeAudio 89
4.17.5 FR-60: PodcastSession.ApplyFilters 89
4.17.6 FR-61: PodcastSession.SaveEditedRecording 90
4.17.7 FR-62: PodcastSession.UpdateFinalAudio 90
4.18 UC-5.3 Podcast Review Before Saving 91
4.18.1 FR-63: PodcastSession.PreviewFinalAudio 91
4.18.2 FR-64: PodcastSession.AdjustVolumeLevels 91
4.18.3 FR-65: PodcastSession.AddEpisodeMetadata 91

4.19 UC-5.4 Export and Save Podcast 92
4.19.1 FR-66: PodcastSession.PublishPodcast 92
4.19.2 FR-67: PodcastSession.GenerateFileName 92
4.19.3 FR-68: PodcastSession.ExportAudio 93
4.19.4 FR-69: PodcastSession.SaveDraft 93
4.20 UC-6.1 AI Host & AI Guest Interaction 94
4.20.1 FR-70: PodcastAI.InitializeSession 94
4.20.2 FR-71: PodcastAI.LoadSettings 94
4.20.3 FR-72: PodcastAI.HostIntroduction 95
4.20.4 FR-73: PodcastAI.GuestResponses 95
4.20.5 FR-74: PodcastAI.ManageTransitions 96
4.20.6 FR-75: PodcastAI.GenerateSummary 96
4.21 UC-7.1 Select Avatars for Speakers 97
4.21.1 FR-76: PodcastAvatars.DisplayPredefinedAvatars 97
4.21.2 FR-77: PodcastAvatars.UploadCustomAvatar 97
4.21.3 FR-78: PodcastAvatars.AssignAvatarsToSpeakers 97
4.21.4 FR-79: PodcastAvatars.ModifyAvatars 98
4.21.5 FR-80: PodcastAvatars.ValidateAvatarSelection 98
4.22 UC-7.2 Select Visual Elements for Video Podcast 99
4.22.1 FR-81: PodcastVisuals.DisplayPredefinedVisuals 99
4.22.2 FR-82: PodcastVisuals.UploadCustomVisuals 99
4.23 UC-7.3 Preview and Export Video Podcast 100
4.23.1 FR-83: PodcastPreview.PlayVideo 100
4.23.2 FR-84: PodcastPreview.ConfirmQuality 100
4.23.3 FR-85: PodcastExport.ExportVideo 101
4.24 UC-7.4 Request Podcast Summary 101
4.24.1 FR-86: PodcastSummary.GenerateSummary 101
4.24.2 FR-87: PodcastSummary.CustomizeSummaryFormat 102
4.25 UC-7.5 Search for Specific Information in a Podcast Summary 102
4.25.1 FR-88: PodcastSummary.SearchKeywords 102
4.25.2 FR-89: PodcastSummary.HighlightResults 103
4.26 UC-7.6 Rate Podcast Session 103
4.26.1 FR-90: PodcastRating.ProvideRating 103
4.26.2 FR-91: PodcastRating.PreventDuplicateRatings 104
4.27 UC-8.1 Subscription Management 104
4.27.1 FR-92: Subscription.DisplayPlans 104
4.27.2 FR-93: Subscription.ProcessPayment 105
4.27.3 FR-94: Subscription.ManageAutoRenewal 105
4.27.4 FR-95: Subscription.HandlePaymentFailure 105
4.28 UC-8.2 Free Content Access Limit 106
4.28.1 FR-96: FreeAccess.TrackUsage 106
4.28.2 FR-97: FreeAccess.PromptSubscription 106
4.29 UC-8.3 Subscription Expiry Reminder 107
4.29.1 FR-98: Subscription.SendExpiryNotification 107
4.29.2 FR-99: Subscription.RenewSubscription 107
4.30 UC-9.1 Ask a Question from FAQs Chatbot 108
4.30.1 FR-100: FAQsChatbot.OpenChatbot 108
4.30.2 FR-101: FAQsChatbot.ProcessUserQuery 108
4.30.3 FR-102: FAQsChatbot.HandleNoMatch 109
4.30.4 FR-103: FAQsChatbot.ImproveResponses 109
4.31 UC-9.2 View Suggested FAQs 110
4.31.1 FR-104: FAQsChatbot.DisplayCommonQuestions 110
4.31.2 FR-105: FAQsChatbot.AllowFAQSelection 110
4.31.3 FR-106: FAQsChatbot.HandleNoFAQsAvailable 111
4.32 UC-10.1 Receive Notifications for Audience Engagement 111

4.32.1 FR-107: Notifications.GenerateEngagementNotification 111
4.32.2 FR-108: Notifications.StoreEngagementHistory 112
4.33 UC-10.2 User-to-User Messaging and Calls 112
4.33.1 FR-109: Messaging.SendMessage 112
4.33.2 FR-110: Messaging.InitiateCall 113
4.34 UC-10.3 Engage with Podcasts (Like, Comment, Share) 113
4.34.1 FR-111: PodcastEngagement.LikePodcast 113
4.34.2 FR-112: PodcastEngagement.CommentOnPodcast 114
4.35 UC-10.4 Provide User Feedback 114
4.35.1 FR-113: Feedback.CollectPodcastRating 114
4.35.2 FR-114: Feedback.SubmitWrittenFeedback 115
5. Non-Functional Requirements 115
5.1 Usability 115
5.2 Perfomance 115
5.3 Security 116
5.4 Interoperability 116
5.5 Integrity 116
6. External Interface Requirements 116
6.1 User Interfaces Requirements 116
6.2 Software interfaces 116
6.3 Hardware interfaces 117
6.4 Communications interfaces 117
7. References 117

1. Introduction

PodVerse is an innovative working platform for small business owners, entrepreneurs, and content creators to create and produce effortless and smooth podcasts. Overcoming issues of costly productions, technical complexities, and getting credible guests, the PodVerse can now enable users to make quality podcasts easily without extensive technical skills. It automates key processes in voice cloning, editing, and publishing, thus making the production process easy. According to AI voices and language models, the user will receive an interesting and dynamic podcast in his or her possession, rather than hosts or guests. The platform is built around allowing creators to work towards creating consistent quality content through this product, helping them grow their brand and effectively engage with the right audience.

1.1 Purpose

PodVerse is a comprehensive solution designed to enhance the podcast creation process, providing a centralized platform for creators to manage their content, engage with their audience, and streamline production. PodVerse offers features such as user authentication, personalized creator dashboards, voice cloning, and knowledge base setup for AI-driven interactions. Creators can easily simulate dynamic hosts and guests, record and edit podcasts, and publish their content seamlessly. By leveraging advanced technologies such as AI and real-time communication, PodVerse aims to automate and optimize various aspects of podcast production, ensuring high-quality and engaging content. With a focus on real-time feedback and audience engagement, PodVerse aims to foster a vibrant podcasting community.

1.2 Scope

PodVerse is a comprehensive solution designed to simplify and enhance the podcast creation process for business owners, entrepreneurs, content creators, and journalists. By automating key aspects of production, PodVerse allows users to focus on content creation while leveraging advanced AI functionalities to handle technical tasks. The platform includes features such as voice cloning, real-time podcast recording, and editing, which help solo creators overcome common barriers like finding reliable guests and coordinating schedules.
PodVerse offers customizable AI avatars, pre-designed video templates, and a robust content management system, enabling users to create both audio and enhanced video podcasts. Its user-friendly design caters to individuals of all technical skill levels, while advanced customization options are available for more experienced users. The platform also includes user profiles and engagement features, allowing users to create and personalize their profiles, follow favorite creators, and interact

through likes, comments, and shares. An integrated communication platform supports user-to-user messaging and calls, fostering a vibrant community.
With its user-centric approach, PodVerse ensures high-quality content creation, making podcasting a more viable and practical medium for brand-building and audience engagement.

1.3 Modules

1.3.1 Module 1: User Access & Profile Management

This module manages authentication, profile customization, and dashboard access for secure login and content engagement.

FE-1: Account creation via email or social media with verification.
FE-2: Secure login via email/password or social login.
FE-3: Password recovery via email verification.
FE-4: Secure logout.
FE-5: Creator's dashboard for sample podcasts and AI Agent management.
FE-6: View and edit profiles.
FE-7: Activity feed with updates from followed creators.

1.3.2 Module 2: Voice Cloning

This module allows users to create and manage voice samples for cloning.

FE-1: Enabling the recording or upload of voice samples for cloning.
FE-2: Allowing users to preview voice samples to check on the quality.
FE-3: Allow users to re-record or re-upload if the sample is not as desired.
FE-4: Allow the availability of a library with AI-generated voices.

1.3.3 Module 3: Knowledge Base Setup

This module facilitates the setup and management of a knowledge base for AI retrieval.

FE-1: Allow users to upload data in PDFs, URLs, CSV, or JSON.
FE-2: Data is cleaned, processed, and split into meaningful chunks for easy handling.
FE-3: Tag chunks with metadata and link to original sources.
FE-4: Store chunks in a database and generate vector embeddings for better AI retrieval.

1.3.4 Module 4: RAG Retrieval

This module enhances the retrieval of accurate and relevant information from the knowledge base.

FE-1: Convert queries into vectors to determine purpose.
FE-2: Match vectors with knowledge base data for accurate responses.
FE-3: Re-rank data by relevance for better answers.
FE-4: Provides fact-based correct answers for podcasts.
FE-5: Optimized for large datasets without loss of performance.
FE-6: Allow feedback to enhance AI response quality.

1.3.5 Module 5: Agent Customization

This module allows users to personalize their AI agents.

FE-1: Users can assign a custom name to their AI agent.
FE-2: Users can choose from various voice options to suit their podcast.
FE-3: Users can select or upload an avatar to visually represent their AI agent.

1.3.6 Module 6: Podcast Audio Creation

This module provides tools for creating and publishing podcasts.

FE-1: Let users start dialogues with a customized AI agent.
FE-2: Conversations recorded for user and AI.
FE-3: Users allowed minimal editing to trim and splice to rearrange things.
FE-4: Allow users the ability to preview edited podcasts for flow and polish.
FE-5: Export podcasts in MP3/WAV.
FE-6: Save recordings for future use.

1.3.7 Module 7: AI Host

This module provides an AI host to manage the flow of podcasts.

FE-1: AI Host presents the flow of podcasts, introduces the episodes and closes with a summary.
FE-2: Tracks podcast time and gives reminders for the schedule.
FE-3: Facilitates smooth transitions between guests or topics.
FE-4: Provides summary prompts to summarize key points for clarity and engagement.

1.3.8 Module 8: AI Guest

This module provides an AI guest to enhance podcast discussions.

FE-1: AI Guest responds to real-time queries using dynamic responses.
FE-2: AI Guest shares appropriate stories to add depth to the discussion.
FE-3: AI Guest gives back comments/reactions to facilitate engagement.

1.3.9 Module 9: Podcast Video Creation & Publishing

This module enables the creation of video content for podcasts, and AI-generated text summaries.

FE-1: Users select avatars for each of the speakers.
FE-2: The system syncs based on the audio of the podcast.
FE-3: Users can choose from the pre-defined templates and visuals for the video.
FE-4: Pre-designed split-screen video template with two avatars.
FE-5: Exports the final video in MP4 format or publish the final video.
FE-6: Allow creators to rate podcast sessions.
FE-7: Provide written feedback on conversation skills, voice clarity, and AI accuracy.

1.3.10 Module 10: Subscription & Payment

This module manages subscription and payment processes.

FE-1: Let users access some free, limited content (predetermined number of podcasts).
FE-2: After the limit, users are prompted to subscribe for continued access.
FE-3: Secure payment gateway integration supporting credit/debit cards.
FE-4: User can control subscriptions, see payment history, change methods.
FE-5: Give reminders before expiration with options of auto-renewal.

1.3.11 Module 11: FAQs Chatbot

This module provides a chatbot to answer frequently asked questions.

FE-1: Users can ask about podcasting, AI customization, or troubleshooting.
FE-2: The chatbot retrieves relevant answers from the FAQs and support materials.
FE-3: Provides concise, direct responses for quick problem resolution.

1.3.12 Module 12: Audience Engagement and Communication

This module enables audience interaction, providing a communication platform, and podcast feedback to enhance creator engagement.

FE-1: Notifications for creators about audience interactions.
FE-2: Highlighting top creators and trending content.
FE-3: Integrated communication platform for user-to-user messaging and calls.
FE-4: Generate and display a text-based summary of podcast episodes.

1.4 Overview

This document describes PodVerse, detailing user interactions with the system and its responses. It also outlines the implementation technologies used in development and highlights the quality attributes associated with PodVerse. Finally, it provides an overview of all interfaces related to the system, ensuring a clear understanding of its functionalities and user experience.

2. Overall Description

2.1 Product Perspective

PodVerse is a new application designed to revolutionize podcast creation and content management. Unlike updates to existing tools, PodVerse is a fresh offering that simplifies and enhances the podcasting process. By leveraging advanced AI technologies, it automates key production tasks, allowing creators to focus on content. PodVerse aims to empower content creators, streamline the podcasting process, and ensure high-quality content creation and effective audience engagement. It fills a gap in the market by offering an all-in-one solution that makes podcasting more accessible and efficient.

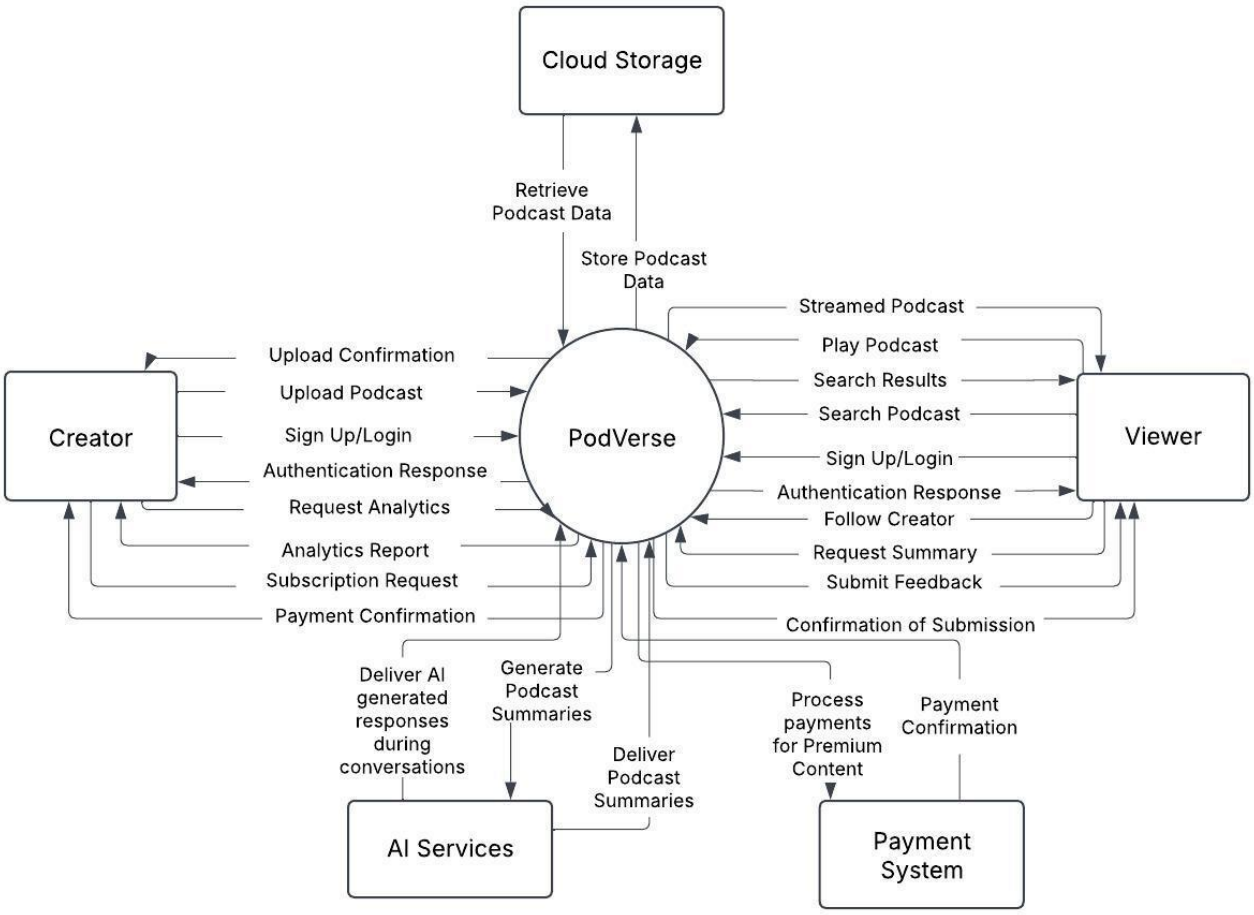


Figure 1 : Context Diagram of System

2.2 User classes and characteristics

User class	Description
Creator	The Creator is a user who records, edits, and publishes podcasts. Each Creator has their own **channel**, where they can post episodes, manage content, and customize their AI hosts/guests. They can track performance analytics such as views, shares, engagement, and listener demographics through the **Channel Insights Dashboard**. Creators can also utilize AI-powered tools like voice cloning and language translation to enhance their podcasts.

Viewer	The Viewer is a user who consumes podcast content but **does not have a channel**. They can follow Creators, engage through likes, comments, and shares, and receive notifications about new episodes. Viewer can explore trending podcasts, search for topics of interest, and provide feedback on episodes. While they do not create content, they contribute to community engagement and content discovery.

Table 1 : User Classes

2.3 Operating Environment

OE-1: PodVerse will operate on modern web browsers such as Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.
OE-2: It will be compatible with major operating systems, including Windows, macOS, iOS, and Android.

2.4 Design and Implementation Constraints

CON-1: The system shall use the MERN stack, consisting of MongoDB, Express.js, React, and Node.js, for the web application's development.

CON-2: The web application shall use the latest stable version of React for front-end development, which is React 18.

CON-3: The backend functionality shall be developed using Node.js, as it complements the MERN stack and allows for easy integration with React on the front end.

CON-4: For AI/ML model integration, Python Scikit-learn and Flask shall be used.

CON-5: TensorFlow and Google Colab shall be used for model training.

CON-6: All user authentication and sensitive data must be handled through secure protocols (e.g., HTTPS) and stored using industry-standard encryption methods to ensure user privacy and data security.

CON-7: Integration with third-party APIs for features like payment processing shall be limited to established providers such as Stripe or PayPal to ensure reliability and security in financial transactions.

3. Requirement Identifying Technique

This section describes the techniques that identify PodVerse's requirements in such a way that it maps a way for PodVerse Final Document better understanding of its functional and non-functional requirements.

3.1 Use Case Diagram

Below is the use case diagram of the Creator (Figure 2) Viewer (Figure 3)

3.1.1 Creator

Below is the use case diagram of Actor-Creator

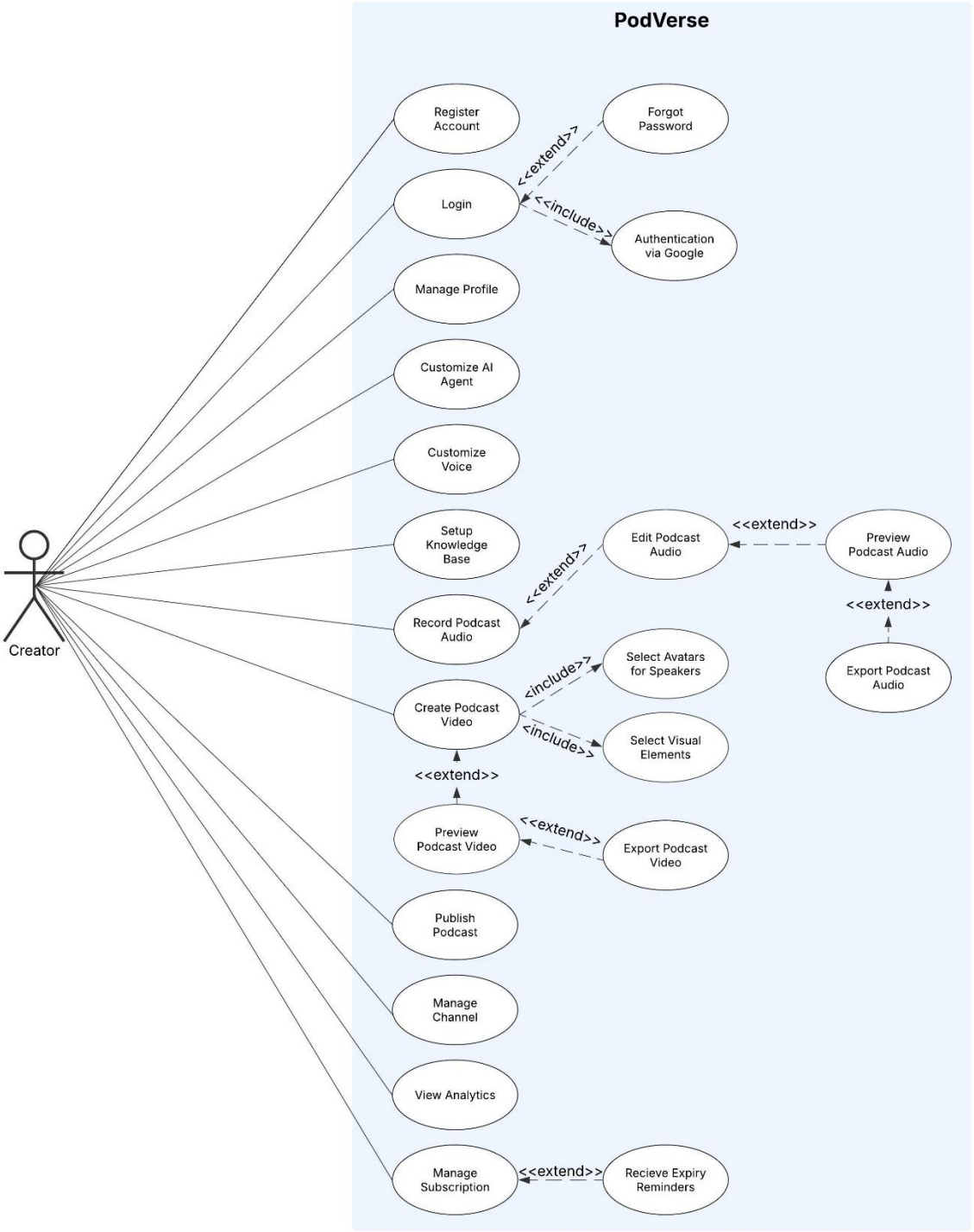


Figure 2 : Use Case Diagram of Creator

3.1.2 Viewer

Below is the use case diagram of Actor-Viewer

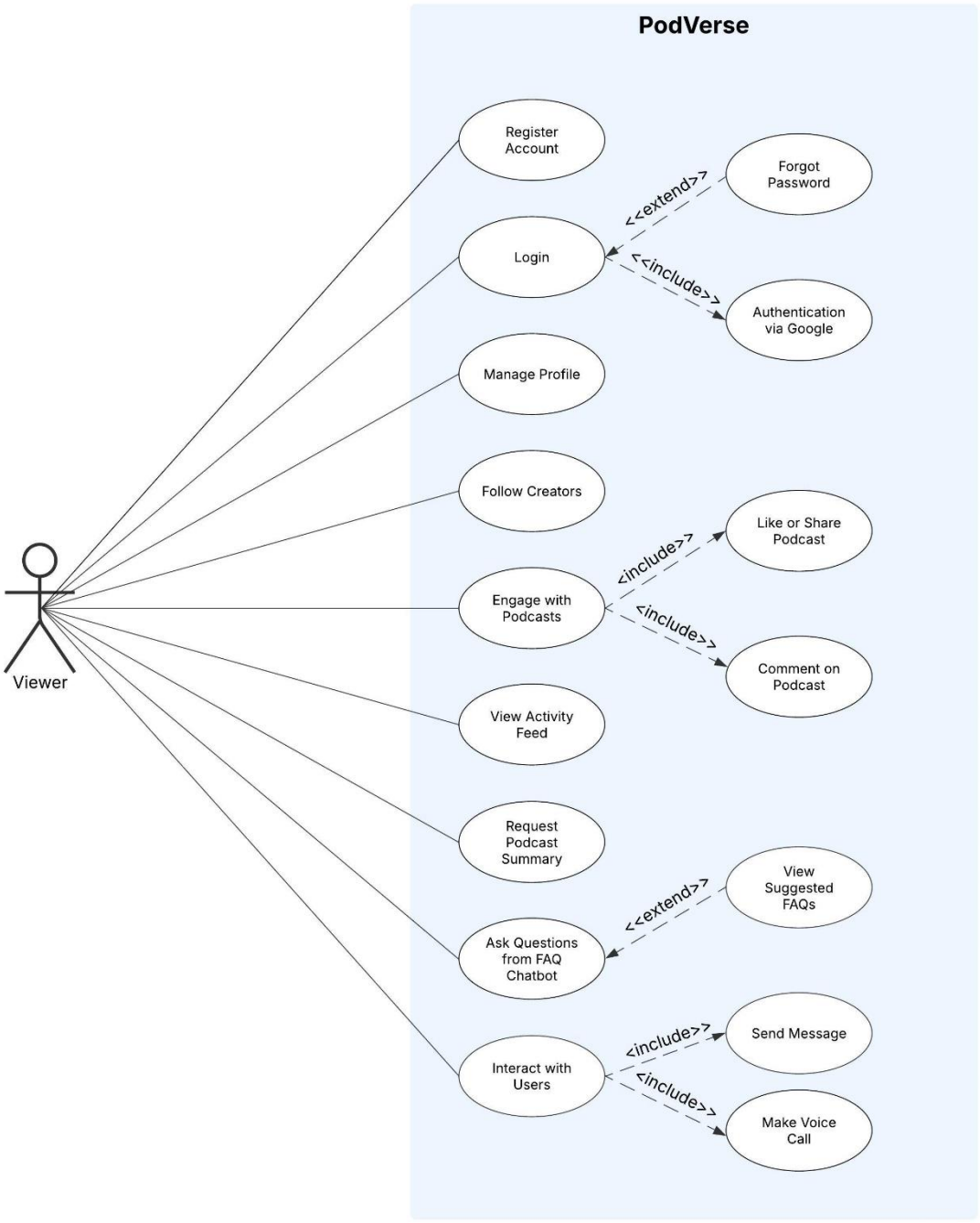


Figure 3 : Use Case Diagram of Viewer

3.2 Business Rules

3.3 Detail Use Cases

3.3.1 Module 1 : User Access & Profile Management

UC-1.1 : Register Account

In this use case, a user registers via email/password or social login. The system ensures email uniqueness and password strength, requiring email verification for email-based sign-ups, while social logins bypass this step for seamless access.

Actors:	User
Trigger:	User selects the **"Sign Up"** option from the login page.
Preconditions:	**PRE-1:** User has a valid email. **PRE-2:** User has an internet connection.
Postconditions:	**POST-1:** User account is created and stored. **POST-2:** A verification email is sent. **POST-3:** User can log in after verifying email.
Normal Flow:	**1.0 Register Account** 1. User selects the "Sign Up" option. 2. User enters email & password. (see 1.0. E1, 1.0. E2) 3. System checks uniqueness & password strength. 4. If valid, system creates the account and sends a verification email. (see 1.0. E3) 5. User verifies the email. 6. User is redirected to the dashboard.

Alternative Flows:	**1.1 Register via Social Login** 1. User selects Google/Facebook login instead of email/password. 2. System retrieves user details & skips email verification.
Exceptions:	**EX-1: 1.0. E1: Email already in use** System prompts the user to enter a new one. 2a. If user cancels, system terminates registration. 2b. If user enters a new email, use case resumes at step 2. **EX-2: 1.0. E2: Weak password entered** System prompts user to meet security criteria. **EX-3: 1.0. E3: Verification link expired** System allows resending a new link.
Business Rules:	**BR-1:** Email must be unique. **BR-2:** Password must meet security criteria. **BR-3:** Email verification is mandatory before login.
Assumptions:	1. User has a working email. 2. OAuth login is configured properly.

Table 2 : UC-1.1 Register Account

UC-1.2 : User Login

A registered user logs in using email/password or social login (Google/Facebook). The system verifies credentials and grants access to the dashboard. If authentication fails, appropriate error messages are shown, and multiple failed attempts may trigger a temporary lock.

Actors:	User
Trigger:	User selects the **"Log In"** option from the login page.

Preconditions:	**PRE-1:** User must have a registered account. **PRE-2:** If using email/password, account must be verified.
Postconditions:	**POST-1:** User session is created. **POST-2:** User is redirected to the dashboard.
Normal Flow:	**1.0 User Login** 1. User enters email & password. (see 1.0.E1, 1.0.E2) 2. System validates credentials. 3. If valid, system creates a session and logs the user in.
Alternative Flows:	**1.1 Social Login** 1. User logs in via Google/Facebook. 2. System verifies identity and logs user in directly.
Exceptions:	**EX-1: 1.0. E1: Incorrect password** System prompts for a retry. **EX-2: 1.0. E2: Unverified email** System notifies the user to verify the email. **EX-3: 1.0. E3: Account not found** System suggests creating a new account.
Business Rules:	**BR-1:** Only verified accounts can log in. **BR-2:** Multiple failed login attempts should trigger temporary lockout.
Assumptions:	1. User knows their credentials. 2. System is connected to authentication services.

Table 3 : UC-1.2 User Login

UC-1.3 : Google Authentication

A user logs into PodVerse using Google authentication for a seamless and secure login. The system verifies credentials via Google and grants access, eliminating the need for a separate password.

Actors:	User, Google Authentication System
Trigger:	User selects the "Sign in with Google" option on the login page.
Preconditions:	**PRE-1:** User must have a valid Google account. **PRE-2:** System must support Google OAuth authentication.
Postconditions:	**POST-1:** User is authenticated and logged into PodVerse. **POST-2:** Google account is linked for future logins.
Normal Flow:	**1.0 Login with Google** 1. User selects "Sign in with Google". 2. System redirects the user to Google's authentication page. 3. User enters Google credentials. (see 1.0.E1) 4. Google requests permission for authentication. 5. User grants permission. 6. System retrieves user details and logs them in.
Alternative Flows:	**1.1 Auto-login for linked accounts** 1. If the user has previously linked their Google account, they are logged in directly.
Exceptions:	**EX-1: 1.0.E1: Google authentication fails** System displays an error and prompts the user to retry. **EX-2: 1.0.E2: User denies permission** System cancels login and redirects to the login page.

Business Rules:	**BR-1:** Only valid Google accounts can be used for authentication. **BR-2:** If the user has not registered before, Google login creates a new account with retrieved details**.**
Assumptions:	1. User has an active internet connection. 2. Google authentication services are operational.

Table 4 : UC-1.3 Google Authentication

UC-1.4 : Password Recovery

In this use case, a user resets their password using the "Forgot Password" option. The system verifies the entered email and sends a password reset link. The user clicks the link, enters a new password, and the system updates their credentials. If the email is unregistered or the reset link has expired, appropriate error messages are displayed.

Actors:	User
Trigger:	User selects the **"Forgot Password"** option from the login page.
Preconditions:	**PRE-1:** User must have a registered email.
Postconditions:	**POST-1:** A password reset link is sent to the user. **POST-2:** User sets a new password.
Normal Flow:	**1.0 Password Recovery** 1. User clicks "Forgot Password". 2. User enters a registered email. (see 1.0. E1) 3. System sends a password reset link. 4. User clicks the link and enters a new password. (see 1.0. E2) 5. System updates the password and confirms the change.

Alternative Flows:	**1.1 Request New Reset Link** 1. User requests a new reset link if the previous one expired.
Exceptions:	**EX-1: 1.0. E1: Email not registered** 1. System displays an error message. 2a. If user cancels, the process terminates. 2b. If user re-enters an email, resume at step 2. **EX-2: 1.0. E2: Reset link expired** 1. System prompts the user to request a new one.
Business Rules:	**BR-1:** Reset links should expire within 30 minutes. **BR-2:** New password must meet security criteria.
Assumptions:	3. User can access their email. 4. Reset links work only for registered users.

Table 5 : UC-1.4 Password Recovery

UC-1.5: Logout

In this use case, a User can securely log out from their active session in the application. The User will initiate the logout process, which will end their active session and redirect them to the login page.

Actors:	User
Trigger:	User selects log out option on dashboard page.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must be on Dashboard Page.
Postconditions:	**POST-1:** User is logged out of the application.

Normal Flow:	1.0 Initate logout Process 1. User selects the logout option on the dashboard ppage. (see 1.1, 1.2) 2. System displays a confirmation message, “Are you sure you want to logout?” 3. User confirms their intention to logout by selecting “OK”. (see 1.0. E1) 4. System logs the user out and redirects them to login page.
Alternative Flows:	1.1 Cancel logout process 1. User selects the option to cancel during the logout confirmation step. (see 1.0) 2. System does not proceed with the logout and use case terminates. 1.2 No active internet connection 1. User tries to logout without an active internet connection. (see 1.0) 2. System displays a message: "Internet connection is required to log out." 3. System does not proceed with the logout and use case terminates.
Exceptions:	**EX-1: 1.0. E1 Server or system error during logout** 1. System encounters an unexpected error, such as a server issue, during the logout process. 2a. If user attempts to logout again, the system retries the logout process. 2b. Else if user decides not to retry and cancels, the use case terminates.
Business Rules:	**N/A**
Assumptions:	1. An active internet connection is available to the user.

Table 6 : UC-1.5 Logout

UC-1.6: Access Sample Podcasts
A user browses sample podcasts for inspiration before starting their own podcast.

Actors:	User
Trigger:	User selects **"Sample podcasts"** from the dashboard panel.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User has an internet connection.
Postconditions:	**POST-1:** Sample Podcasts are displayed.
Normal Flow:	1. User naivgtes to Sample Podcasts. 2. System loads featured sample podcasts. 3. User plays, pauses, or skip through podcasts.
Alternative Flows:	**AF-1:** User searches for specific sample podcasts using a filter.
Exceptions:	**EX-1:** No Sample podcasts are found.
Business Rules:	**BR-1:** Only verified user can access sample of podcasts.
Assumptions:	User has an active internet connection.

Table 7 : UC-1.6 Access Sample Podcasts

UC-1.7: Start a Podcast Session
A user wants to start a podcast session with either AI guest or AI host .

Actors:	Creator
Trigger:	Creator logs in and selects **"Start New Session".**
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have access to AI Guest or AI Host.
Postconditions:	**POST-1:** User can now specify the settings for podcast session firstly agent customization.
Normal Flow:	1. User naivgtes to Start a New Session. 2. User chooses to continue either with AI Host or AI Guest. 3. User can now set the Agent customization.
Alternative Flows:	If there is an Incomplete Session. 1. Incomplete Session is opened. 2. Resume from the previous saved session.
Exceptions:	**EX-1:** Credits are finished.
Business Rules:	**BR-1:** Only verified user can access sample of podcasts.
Assumptions:	1. User has an active internet connection. 2. User has credits to create a podcast session.

Table 8 : UC-1.7 Start a Podcast Session

UC-1.8: View Dashboard

A user wants to go to Dashboard screen or home screen.

Actors:	User
Trigger:	User selects the home option on dashboard screen.
Preconditions:	**PRE-1:** User must be logged in.
Postconditions:	**POST-1:** User sees a list of relevant posts based on their profile and the people they have followed.
Normal Flow:	1. User selects the the "**home**" option. 2. System retrieves and displays a list of podcasts based on: a. User's own podcasts. b. Posts from users they follow. c. Public posts (depending on platform privacy settings).
Alternative Flows:	N/A
Exceptions:	**EX-1:** No internet connection • System shows an error message. • System tries to reload.
Business Rules:	**BR-1:** Appropriate content moderation policies must be implemented.
Assumptions:	User can locate the home option from nav-bar on Dashboard screen.

Table 9 : UC-1.8 View Dashboard

UC-1.9 : Create and Personlize User Profile

In this use case, a user creates and customizes their profile, including setting a display name, uploading a profile picture, and adding personal details.

Actors:	Creator
Trigger:	User navigates to the profile section for the first time.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must allow users to update profile information.
Postconditions:	**POST-1:** The user profile is created and saved. **POST-2:** User can update their profile at any time.
Normal Flow:	**1.0 Create and Personalize User Profile** 1. User navigates to "My Profile". 2. System prompts the user to enter profile details (display name, bio, profile picture). 3. User enters the required details. (see EX-1) 4. System saves and updates the user profile. 5. Profile is displayed on the user's page.
Alternative Flows:	**1.1 Modify Profile Details** 1. User revisits "My Profile" to update details. 2. System allows changes to profile picture, bio, and personal settings
Exceptions:	**EX-1: Invalid file format for profile picture** 1. System rejects the upload and prompts the user to select a supported format.
Business Rules:	**BR-1:** Users must provide a unique display name. **BR-2:** Profile updates should be saved immediately.

Assumptions:	1. Users will regularly update their profile to reflect their activity.

Table 10 : UC-1.9 Create and Personalize User Profile

UC-1.10: Edit Profile
In this use case, a user wants to modify their profile details. They can update their username, profile picture, or password. Once the changes are made, the system ensures the updated information reflects in the user's profile.

Actors:	User
Trigger:	User selects edit profile option on dashboard page.
Preconditions:	**PRE-1:** User must be logged in.
Postconditions:	**POST-1:** System updates the User information
Normal Flow:	1.0 Edit User Profile 1. User selects edit profile option in dashboard page. 2. User updates phone number if required. 3. User updates username if required. (see 1.0. E1) 4. User updates picture if required. 5. User updates password if required. (see 1.0. E2) 6. System updates the user information.
Alternative Flows:	1.1 Password Updates 1. User enters the new password. 2. User re-enter the password for confirmation. 3. System checks if the two passwords match (see 1.1 E1). 4. If they match, the system updates the password and use case resumes at step 6 of the normal flow.

Exceptions:	**EX-1: 1.0. E1 Username already registered** 1. System notifies the user that the chosen username has already taken. 4. 2a. If the user provides an alternate username, the use case resumes from step 3 of the normal flow. 2b. Else if user decides to exit the registration process, the use case terminates. **EX-2: 1.1. E2 Password Strength Insufficient** 1. System informs user that the password doesn't meet the required criteria. 2a. If the user modifies the password according to guidelines, the use case resumes at step 5 of the normal flow. 2b. Else if user cancels the password update process, the use case resumes at step 6 of the normal flow. **EX-3: 1.1. E1 Password and Confirm Password Mismatch** 2. System notifies User that the entered passwords don't match. 2a. If user re-enters matching passwords, use case resumes at step 6 of the normal flow. 2b. Else if user cancels the password update process, the use case resumes at step 6 of the normal flow.
Business Rules:	**BR-1:** Only one account can be registered for a given username. **BR-2:** The password must be a strong password with both alphabet and digits
Assumptions:	1. User knows that edit profile will change the previous information. 2. An active internet connection is available to the user.

Table 11 : UC-1.10 Edit Profile

UC-1.11 : Follow Creators and View Updates

In this use case, a user follows their favorite podcast creators to receive updates on new episodes and activity.

Actors:	Creator
Trigger:	User selects the "Follow" button on a creator’s profile.

Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The creator's profile must be publicly available.
Postconditions:	**POST-1:** The user starts following the creator. **POST-2:** Updates from the creator appear in the user's feed.
Normal Flow:	**1.0 Follow a Creator** 1. User visits a creator's profile. 2. User clicks "Follow". 3. System updates the user's following list. 4. System adds creator's updates to the user's feed.
Alternative Flows:	**1.1 Unfollow a Creator** 1. User visits a followed creator's profile. 2. User clicks "Unfollow". 3. System removes the creator from the following list.
Exceptions:	**EX-1: Creator's account is deleted or deactivated** 1. System removes the creator from all users' following lists.
Business Rules:	**BR-1:** Users can follow or unfollow creators at any time.
Assumptions:	1. Creators regularly update their content.

Table 12 : UC-1.11 Follow Creators and View Updates

UC-1.12 : View Activity Feed of Followed Creators

In this use case, a user views an feed displaying updates from followed creators.

Actors:	User
Trigger:	User navigates to "Activity Feed" on home page.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must follow at least one creator.
Postconditions:	**POST-1:** System displays updates from followed creators.
Normal Flow:	**1.0 View Activity Feed** 1. User selects "Activity Feed". 2. System loads recent updates from followed creators. 3. User browses through new content, comments, and interactions.
Alternative Flows:	**N/A**
Exceptions:	**EX-1: No activity available** 1. System informs the user and suggests following more creators.
Business Rules:	**BR-1:** Activity feed updates should be in real-time.
Assumptions:	1. Creators actively post updates.

Table 13 : UC-1.12 View Activity Feed of Followed Creators

3.3.2 Module 2 : Voice Cloning

UC-2.1 : Voice Cloning Setup

In this use case, a user records, uploads, or selects a voice sample for AI voice cloning. The system processes the sample, generates a cloned voice, and allows the user to preview and save it. If unsatisfied, the user can re-record, re-upload, or choose from AI-generated voices for customization.

Actors:	Creator
Trigger:	User selects the "Voice Cloning" option from the settings or AI customization menu.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must support voice recording and AI processing.
Postconditions:	**POST-1:** A voice sample is recorded, uploaded, or selected. **POST-2:** The system processes and saves the voice model.
Normal Flow:	**1.0 Voice Cloning Setup** 1. User selects "Voice Cloning" from settings. 2. System prompts the user to record or upload a voice sample. 3. User records or uploads a sample. (see 1.0. E1, 1.0. E2) 4. System processes the sample for cloning. 5. User previews the generated AI voice. 6. If satisfied, user confirms and saves the AI-generated voice.
Alternative Flows:	**1.1 Modify Cloned Voice** 1. If the user is not satisfied, they can re-record or re-upload a sample. (see 1.1. E1) 2. User can select a voice from the AI-generated voice library instead of recording their own.

Exceptions:	**EX-1: 1.0. E1: Poor-quality recording** 1. System prompts the user to retry. **EX-2: 1.0. E2: Unsupported file format for upload** 1. System notifies the user and requests a valid format. **EX-3: 1.1. E1: User rejects generated voice** 1. System allows re-recording, re-uploading, or selection from the AI-generated voice library.
Business Rules:	**BR-1:** Voice cloning should meet quality standards before being saved. **BR-2:** Users should be able to re-record or modify their cloned voice.
Assumptions:	1. User has a functioning microphone if recording a voice sample. 2. The system has AI capabilities to process and generate voices.

Table 14 : UC-2.1 Voice Cloning Setup

3.3.3 Module 3 : Knowledge Base Setup

UC-3.1 : Knowledge Base Setup

In this use case, a user uploads structured or unstructured data into the AI system's knowledge base. The system processes, cleans, and organizes the data into meaningful chunks for better AI retrieval. Metadata is attached to ensure accurate responses, and the final processed data is stored in a vector database.

Actors:	User, System
Trigger:	User selects the "Upload Knowledge Base" option from the settings or AI customization menu.

Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have valid data sources in supported formats (PDFs, URLs, CSV, JSON). **PRE-3:** System must have available storage for data processing.
Postconditions:	**POST-1:** Uploaded data is processed and stored. **POST-2:** Data is split into meaningful chunks with metadata attached. **POST-3:** System generates vector embeddings for efficient retrieval.
Normal Flow:	**1.0 Knowledge Base Upload** 1. User selects "Upload Knowledge Base". 2. System prompts the user to choose a data source (PDF, URL, CSV, JSON). 3. User uploads the file or enters a URL. (see 1.0. E1) 4. System scans the data for formatting issues. 5. System cleans and preprocesses data. 6. Data is split into structured chunks with metadata. 7. System generates vector embeddings for AI retrieval. 8. Processed data is stored in the vector database. 9. System confirms successful upload.
Alternative Flows:	**1.1 Metadata Enhancement** 1. System detects missing metadata. (see 1.1. E1) 2. System suggests default metadata values. 3. User manually updates metadata if necessary.
Exceptions:	**EX-1: 1.0. E1: File format unsupported** 1. System notifies the user and requests a valid format. **EX-2: 1.0. E2: URL is unreachable** 2. System notifies the user and allows them to enter another URL. **EX-3: 1.1. E1: Missing metadata** 3. System prompts the user to add metadata before proceeding.
Business Rules:	**BR-1:** Only supported file formats (PDF, URL, CSV, JSON) can be processed. **BR-2:** Data must be cleaned and structured before storage. **BR-3:** Metadata is required for AI retrieval accuracy.

Assumptions:	1. User uploads valid and well-formatted data. 2. System can process large datasets efficiently.

Table 15 : UC-3.1 Knowledge Bas Setup

3.3.4 Module 5 : Agent Customization

UC-4.1 : AI Agent Customization

In this use case, a user customizes their AI agent before starting a podcast session. The system allows users to configure the agent's name, voice, and avatar to match their podcast style.

Actors:	User, System
Trigger:	User selects "Customize AI Agent" after starting a new podcast session.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** AI customization feature must be enabled.
Postconditions:	**POST-1:** AI agent is configured with a selected name, voice, and avatar. **POST-2:** The customized agent is saved for future sessions.
Normal Flow:	**1.0 AI Agent Customization** 1. User selects "Customize AI Agent". 2. System prompts the user to enter a name for the AI agent. (see 1.0. E1) 3. User uploads or selects an avatar for the AI agent. (see 1.0. E2) 4. System applies selected settings. 5. User confirms the customization and saves changes.

Alternative Flows:	**1.1 Skip Customization** 1. User chooses to proceed with default AI settings. 2. System loads default AI agent configuration.
Exceptions:	**EX-1: 1.0. E1: AI name not provided** 1. System prompts the user to enter a name. **EX-2: 1.0. E2: Invalid avatar file format** 1. System requests a valid file format.
Business Rules:	**BR-1:** AI agent customization must be completed before recording begins. **BR-2:** Users can modify AI settings before each session.
Assumptions:	1. System provides a variety of AI voice and avatar options. 2. Users have permission to customize AI settings.

Table 16 : UC-4.1 AI Agent Customization

3.3.5 Module 6 : Podcast Audio Creation

UC-5.1 : Record Podcast Session

In this use case, a user records a podcast session with a customized AI agent. The system captures real-time audio, records AI-generated responses, and generates a live transcript. Users can pause, stop, or resume recording at any time, ensuring flexibility in content creation.

Actors:	User
Trigger:	User clicks "Start Recording" after customizing the AI agent.
Preconditions:	**PRE-1:** User has configured AI agent settings. **PRE-2:** User has selected AI Host or AI Guest. **PRE-3:** User is on the recording page.

Postconditions:	**POST-1:** A recording session is started. **POST-2:** Audio is recorded in real-time. **POST-3:** AI-generated responses are recorded in the same file. **POST-4:** Transcript is generated alongside the recording.
Normal Flow:	**1.0 Record Podcast Session** 1. User clicks Start Recording. 2. System records the user's voice and AI-generated responses in real-time. (see 1.0. E1) 3. System generates a real-time transcript of the conversation. 4. User has the option to pause or stop recording at any time. 5. When the recording is complete, the system displays a confirmation message.
Alternative Flows:	**1.1 Pause and Resume Recording** 1. If the user pauses the recording, the system allows resuming later.
Exceptions:	**EX-1: 1.0. E1: User's microphone access is denied** 1. System prompts for permissions. **EX-2: 1.0. E2: Storage is full** 1. System warns user and suggests freeing space. **EX-3: 1.0. E3: AI response failure** 1. System retries generating an AI response.
Business Rules:	**BR-1:** AI-generated voice must be synchronized with user input. **BR-2:** The system must auto-save progress at intervals.
Assumptions:	1. User's device has a working microphone. 2. AI voice models are pre-loaded before recording.

Table 17 : UC-5.1 Record Podcast Session

UC-5.2 : Edit Podcast Recording

In this use case, a user edits a recorded podcast using trimming, splicing, and rearranging tools. The system allows the user to preview changes before saving the final version.

Actors:	User
Trigger:	User selects "Edit Recording" after finishing the session.
Preconditions:	**PRE-1:** A recorded podcast session exists. **PRE-2:** User has completed the recording.
Postconditions:	**POST-1:** The user edits the audio using trimming and splicing options. **POST-2:** The edited audio is saved.
Normal Flow:	**1.0 Edit Podcast Recording** 1. User selects Edit Recording. 2. System loads the audio editing interface. (see 1.0. E1) 3. User trims and rearranges audio clips. 4. User listens to a preview of the edited audio. 5. User saves the edited recording. 6. System updates the final audio file.
Alternative Flows:	**N/A**
Exceptions:	**EX-1: 1.0.E1: Editing tools fail to load** 1. System retries loading. **EX-2: 1.0.E2: Audio file is corrupted** 1. System provides an error message.

Business Rules:	**BR-1:** Editing should not degrade audio quality. **BR-2:** System must auto-save edits periodically.
Assumptions:	1. User has basic knowledge of audio editing tools. 2. User is satisfied with the recording before editing.

Table 18 : UC-5.2 Edit Podcast Recording

UC-5.3 : Podcast Review Before Saving

In this use case, a user previews a finalized podcast recording before saving to ensure content quality. The system allows the user to listen to the entire podcast and make final adjustments before confirming.

Actors:	User
Trigger:	User selects Preview Podcast Audio before saving.
Preconditions:	**PRE-1:** A final edited podcast audio exists.
Postconditions:	**POST-1:** The user listens to the podcast. **POST-2:** The user can return to editing if needed. **POST-3:** The podcast is approved for saving.
Normal Flow:	**1.0 Podcast Review Before Saving** 1. User selects "Preview Podcast". 2. System plays the full podcast. (see EX-1) 3. User listens and confirms quality. 4. User either approves for saving or returns to editing.

Alternative Flows:	**1.1 Return to Editing** 1. If the user finds errors, they can return to editing and make changes.
Exceptions:	**EX-1: Audio playback failure** 1. System retries playing.
Business Rules:	**BR-1:** Users must preview before saving.
Assumptions:	1. The system plays the podcast without lag or distortion.

Table 19 : UC-5.3 Podcast Preview Before Saving

UC-5.4 : Export and Save Podcast

In this use case, a user exports their finalized podcast audio and saves it in MP3/WAV format for personal use. The podcast is not published on the platform but can be stored for later use or further processing.

Actors:	User
Trigger:	User selects Export & Save after previewing the podcast.
Preconditions:	**PRE-1:** A finalized podcast recording exists. **PRE-2:** The user has previewed the content.
Postconditions:	**POST-1:** Podcast is exported in MP3/WAV format. **POST-2:** Podcast is saved locally for future use.

Normal Flow:	**1.0 Export and Save Podcast** 1. User selects "Export & Save". 2. System prompts the user to choose a file format (MP3/WAV). (see 1.0. E1) 3. System converts and exports the file. 4. User confirms saving. 5. Podcast is saved locally on the user's device or cloud storage.
Alternative Flows:	**1.1 Save as Draft** 1. If the user does not want to export immediately, they can save the file draft for later.
Exceptions:	**EX-1: 1.0.E1: Export failure due to insufficient storage** 1. System prompts for cleanup. **EX-2: 1.0.E2: File format unsupported** 1. System requests user to select a valid format.
Business Rules:	**BR-1:** Users must confirm before saving. **BR-2:** Exported files must meet quality standards.
Assumptions:	1. Users have enough storage for exporting.

Table 20 : UC-5.4 Export and Save Podcast

3.3.6 Module 7 : AI Host & Module 8 : AI Guest

UC-6.1 : AI Host & AI Guest Interaction

In this use case, a user initiates a podcast session with either an AI Host or AI Guest. The AI Host manages the flow of the podcast, introduces episodes, ensures smooth transitions, and summarizes discussions. The AI Guest responds dynamically to user questions, shares relevant stories, and engages in discussions.

Actors:	Creator, System
Trigger:	User starts a podcast session with AI Host or AI Guest.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have selected AI Host or AI Guest. **PRE-3:** Knowledge base and voice model must be pre-configured.
Postconditions:	**POST-1:** AI Host/Guest interacts with the user in a structured podcast session. **POST-2:** AI-generated responses and podcast flow are recorded. **POST-3:** AI Host provides episode summary upon completion.
Normal Flow:	**1.0 AI Podcast Session** 1. User selects "Start Podcast" and chooses AI Host or AI Guest. 2. System loads pre-configured AI settings, voice model, and knowledge base. 3. AI Host introduces the episode and sets the discussion agenda. (see 1.0. E1) 4. AI Guest engages in conversation by responding to queries and sharing relevant insights. (see 1.0. E2) 5. AI Host facilitates smooth transitions between topics. 6. AI Host provides summary prompts to recap key points. 7. The session concludes, and the recording is saved.
Alternative Flows:	**N/A**
Exceptions:	**EX-1: 1.0. E1: AI Host fails to load settings** 1. System prompts user to reconfigure AI preferences. **EX-2: 1.0. E2: AI Guest fails to generate responses** 1. System retries or defaults to a generic knowledge base.

Business Rules:	**BR-1:** AI Host must ensure structured podcast flow. **BR-2:** AI Guest should generate contextually relevant responses. **BR-3:** AI must provide a coherent and engaging podcast experience.
Assumptions:	1. AI models are trained to deliver engaging and structured conversations. 2. The system has sufficient processing power to generate real-time AI responses.

Table 21 : UC-6.1 AI Host & AI Guest Interaction

3.3.7 Module 9 : Podcast Video Creation & Publishing

UC-7.1 : Select Avatars for Speakers

In this use case, a user selects AI-generated avatars for each speaker in the video podcast. The system provides customization options to match the podcast theme.

Actors:	Creator, System
Trigger:	User selects "Choose Avatars" while creating a video podcast.
Preconditions:	**PRE-1:** The podcast audio is available. **PRE-2:** System-provided avatars and an option to upload a custom image are accessible.
Postconditions:	**POST-1:** Avatars are successfully assigned to each speaker.
Normal Flow:	**1.0 Select Avatars** 1. User navigates to "Choose Avatars". 2. System displays a selection of predefined avatars.

	3. User selects an avatar for each speaker. (see 1.0. E1) 4. If the user wants a custom avatar, they can upload their own. (see 1.0. E2) 5. System applies the chosen avatars to the podcast. 6. User confirms the selection and proceeds.
Alternative Flows:	**1.1 Modify Avatar Selection** 1. User decides to change avatars before finalizing. 2. System allows reselection or re-upload.
Exceptions:	**EX-1: 1.0. E1: User does not select an avatar** 1. System prompts user to choose or upload an avatar. **EX-2: 1.0. E2: Upload failure** 1. System notifies the user and requests a valid format.
Business Rules:	**BR-1:** Users must assign an avatar to each speaker before proceeding. **BR-2:** Uploaded avatars must meet system compatibility standards.
Assumptions:	1. Users have access to predefined avatars and an option to upload.

Table 22 : UC-7.1 Select Avatars for Speakers

UC-7.2 : Select Visual Elements for Video Podcast

In this use case, a user selects or uploads custom visual elements such as backgrounds and overlays for the video podcast. The system provides a library of visuals while allowing user uploads.

Actors:	Creator, System
Trigger:	User selects "Choose Visuals" during video creation.

Preconditions:	**PRE-1:** The podcast audio is available. **PRE-2:** System-provided visuals and an option to upload a custom visuals are accessible.
Postconditions:	**POST-1:** User selects or uploads visuals for the podcast. **POST-2:** System applies the visuals to the video podcast.
Normal Flow:	**1.0 Select Visual Elements** 1. User navigates to "Choose Visuals". 2. System presents a library of background themes, overlays, and effects. 3. User selects visuals for the video podcast. (see 1.0. E1) 4. If the user wants a custom visual, they can upload their own. (see 1.0. E2) 5. System applies the chosen visuals to the podcast. 6. User confirms the selection and proceeds.
Alternative Flows:	**1.1 Modify Visual Selection** 1. User decides to change visuals before finalizing. 2. System allows reselection or re-upload.
Exceptions:	**EX-1: 1.0. E1: User does not select visuals** 1. System prompts user to choose or upload visuals. **EX-2: 1.0. E2: Upload failure** 1. System notifies the user and requests a valid format.
Business Rules:	**BR-1:** Users must assign at least one visual element before proceeding. **BR-2:** Uploaded visuals must meet system compatibility standards.
Assumptions:	1. Users have access to predefined visuals and an option to upload.

Table 23 : UC-7.2 Select Visual Elements for Video Podcast

UC-7.3 : Preview and Export Video Podcast

In this use case, a user previews the video podcast before exporting or publishing. The system plays back the video, allowing the user to review the final product before confirming the export.

Actors:	Creator
Trigger:	User selects "Preview & Export" before publishing.
Preconditions:	**PRE-1:** User must have applied avatars and visuals to the podcast.
Postconditions:	**POST-1:** User reviews and approves the video. **POST-2:** System exports the video in MP4 format.
Normal Flow:	**1.0 Preview and Export** 1. User selects "Preview Video". 2. System plays back the full video. 3. User reviews and confirms the video quality. 4. User selects "Export" and chooses MP4 format. 5. System processes and saves the final video.
Alternative Flows:	**N/A**
Exceptions:	**N/A**
Business Rules:	**BR-1:** Users must preview before exporting.

Assumptions:	1. System ensures smooth playback.

Table 24 : UC-7.3 Preview and Export Video Podcast

UC-7.4 : Request Podcast Summary

In this use case, a user requests an AI-generated summary of a podcast episode. The system processes the episode's content, identifies key topics, and generates a structured text-based summary.

.

Actors:	User
Trigger:	User selects "Request Summary" on a podcast episode.
Preconditions:	**PRE-1:** The podcast episode must exist in the system. **PRE-2:** AI-powered summarization feature must be enabled.
Postconditions:	**POST-1:** System processes and displays a text-based summary. **POST-2:** Summary includes main topics and key highlights.
Normal Flow:	**1.0 Request Podcast Summary** 1. User selects a podcast episode. 2. User clicks the "Request Summary" option. 3. System processes the episode's audio content. 4. System extracts key topics and generates a summary. 5. System displays a structured summary with key points.
Alternative Flows:	**1.1 Customize Summary Format** 1. User chooses to generate a detailed or brief summary. 2. System adjusts the output based on user selection.

Exceptions:	**EX-1: 1.0. E1: Podcast episode not found** 1. System notifies the user and suggests selecting another episode. **EX-2: 1.0. E2: AI fails to generate a summary** 1. System retries or provides a generic summary.
Business Rules:	**BR-1:** Summaries must be factually accurate and concise. **BR-2:** AI should ensure key points are highlighted clearly.
Assumptions:	1. Users prefer quick, AI-generated summaries for reference.

Table 25 : UC-7.4 Request Podcast Summary

UC-7.5 : Search for Specific Information in a Podcast Summary

In this use case, a user searches for specific topics or keywords within a podcast summary. The system highlights relevant parts of the summary and provides timestamped references for easy navigation.

.

Actors:	User
Trigger:	User enters a keyword or topic in the summary search bar.
Preconditions:	**PRE-1:** A podcast summary must be generated and available. **PRE-2:** System must support keyword-based searching.
Postconditions:	**POST-1:** System filters and highlights relevant summary content. **POST-2:** System provides timestamped references to key moments.
Normal Flow:	**1.0 Search for Topics in a Summary** 1. User navigates to a podcast summary. 2. User enters a keyword or topic in the search bar. 3. System scans the summary for relevant content. 4. System highlights matching sections.

	5. System provides timestamped references for easy navigation.
Alternative Flows:	**1.1 Filter Search Results** 1. User applies filters (e.g., most relevant, newest content). 2. System adjusts the results accordingly.
Exceptions:	**EX-1: No matching content found** 1. System suggests related topics or keywords.
Business Rules:	**BR-1:** Search results must be accurate and relevant. **BR-2:** Users should be able to navigate directly to timestamped moments.
Assumptions:	1. Users search summaries to quickly find specific information.

Table 26 : UC-7.5 Request Podcast Summary

UC-7.6 : Rate Podcast Session

In this use case, a user searches for specific topics or keywords within a podcast summary. The system highlights relevant parts of the summary and provides timestamped references for easy navigation.
.

Actors:	Creator
Trigger:	The user decides to rate a completed podcast session.
Preconditions:	**PRE-1:** The podcast session must be completed. **PRE-2:** The user must be logged in.

Postconditions:	**POST-1:** The rating is recorded and associated with the podcast session. **POST-2:** The system updates the session's overall rating.
Normal Flow:	**1.0 Rate the session** 1. The user navigates to the completed podcast session. 2. The system displays the rating options (e.g., 1 to 5 stars). 3. The user selects a rating. 4. The system saves the rating and updates the session's rating statistics. 5. A confirmation message is displayed.
Alternative Flows:	**1.1 User Cancels Rating** 1. The user chooses not to rate. 2. The system does not save any rating.
Exceptions:	**EX-1: Session Not Found** If the session is deleted or unavailable, the system notifies the user. **EX-2: Duplicate Rating Attempt** If the user tries to rate the same session twice, the system notifies them that the rating was already submitted.
Business Rules:	**BR-1:** Users can rate only sessions they have hosted. **BR-2:** Ratings should be final once submitted.
Assumptions:	Users will rate sessions to provide meaningful feedback.

Table 27 : UC-7.6 Rate Podcasts Session

3.3.8 Module 10 :Subscription and Payment

UC-8.1 : Subscription Management

In this use case, a user manages their subscription, including subscribing, canceling, or renewing their plan. The system processes payments securely and updates the user’s access accordingly.

Actors:	User, System
Trigger:	User navigates to the **Subscription & Payments** section.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must have an active payment gateway.
Postconditions:	**POST-1:** Subscription is updated based on user actions. **POST-2:** User gains or loses access to premium features accordingly.
Normal Flow:	**1.0 Subscription Management** 1. User navigates to "Subscription & Payments". 2. System displays available subscription plans. 3. User selects a plan and enters payment details. 4. System processes payment and updates the subscription. 5. User receives a confirmation notification.
Alternative Flows:	**1.1 Modify Subscription** 1. User cancels their subscription. (see 1.1. E1) 2. System stops auto-renewal and schedules access termination. 3. User updates payment method. (see 1.1. E2) 4. System verifies the new details and applies changes.
Exceptions:	**EX-1: 1.1. E1: Payment fails** 1. System prompts user to retry or update payment method. **EX-2: 1.1. E2: Subscription renewal fails** 1. System notifies the user and requests manual action.
Business Rules:	**BR-1:** Users must have an active subscription for premium features. **BR-2:** Subscription auto-renewal settings should be user-configurable.

Assumptions:	1. System supports secure payment processing.

Table 28 : UC-8.1 Subscription Management

UC-8.2 : Free Content Access Limit

In this use case, a user accesses free content until they reach the predetermined limit, after which they are prompted to subscribe.

Actors:	User, System
Trigger:	User access free content (e.g., record limited number of podcasts).
Preconditions:	**PRE-1:** User has not exceeded the free content limit.
Postconditions:	**POST-1:** User either continues listening or is prompted to subscribe.
Normal Flow:	**1.0 Free Content Access Limit** 1. User records a free podcast. 2. System tracks free content usage. (see EX-1) 3. If limit is not reached, user continues listening. 4. If limit is reached, system displays a subscription prompt.
Alternative Flows:	**1.1 Subscription Prompt** 1. User ignores the prompt and exits. 2. User subscribes and gains unlimited access.

Exceptions:	**EX-1: Content playback failure** 1. System retries loading.
Business Rules:	**BR-1:** Free users have a limited number of content views.
Assumptions:	1. System tracks user progress accurately.

Table 29 : UC-8.2 Free Content Access Limit

UC-8.3 : Subscription Expiry Reminder

In this use case, the system **notifies users** before their subscription expires and provides renewal options.

Actors:	User, System
Trigger:	System detects an upcoming subscription expiration.
Preconditions:	**PRE-1:** User has an active subscription.
Postconditions:	**POST-1:** User is reminded about the expiration. **POST-2:** User can renew or cancel the subscription.
Normal Flow:	**1.0 Subscription Expiry Reminder** 1. System detects expiration date approaching. 2. System sends reminder notifications. (see EX-1) 3. User chooses to renew, cancel, or ignore. 4. System updates the subscription based on user action.

Alternative Flows:	**1.1 Renewal Process** 1. User renews the subscription before expiry. 2. System processes payment and extends access.
Exceptions:	**EX-1: Payment failure during renewal** 1. System prompts the user to update payment details.
Business Rules:	**BR-1:** Users must receive reminders before expiration.
Assumptions:	1. System has a working notification service.

Table 30 : UC-8.3 Subscription Expiry Reminder

3.3.9 Module 11: FAQs Chatbot

UC-9.1 : Ask a Question from FAQs Chatbot

In this use case, a user interacts with the FAQs chatbot to ask questions related to podcasting, AI customization, or troubleshooting. The chatbot searches the FAQs database and provides the most relevant answers.

Actors:	User
Trigger:	User selects the FAQs chatbot option from the left panel and enters a query.

Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The FAQs database must be available and populated with relevant content.
Postconditions:	**POST-1:** The chatbot retrieves and displays an answer from the FAQs database. **POST-2:** The system logs the interaction to improve future chatbot responses.
Normal Flow:	**1.0 Ask a Question from FAQs Chatbot** 1. User opens the FAQs chatbot from the menu. 2. User types a question related to podcasting, AI customization, or troubleshooting. 3. System processes the query and searches the FAQs database. 4. System retrieves the most relevant response. (see 1.0. E1) 5. Chatbot displays the answer in a structured format. 6. User can either close the chatbot or ask another question.
Alternative Flows:	**1.1 No Exact Match Found** 1. If no exact match is found, the chatbot suggests related FAQs. (see 1.1. E1) 2. User selects the most relevant suggestion.
Exceptions:	**EX-1: 1.0. E1: No data available in FAQs database** 1. System informs the user that no answer is available. 2. System provides an option to contact support. **EX-2: 1.1. E1: User enters an unclear or incomplete query** 1. Chatbot asks for clarification.
Business Rules:	**BR-1:** Chatbot should only display verified responses from the FAQs database. **BR-2:** Chatbot should continuously improve responses based on user interactions.
Assumptions:	1. Users will ask relevant questions. 2. The FAQs database is regularly updated.

Table 31 : UC-9.1 Ask a Question from FAQs Chatbot

UC-9.2 : View Suggested FAQs

In this use case, a user browses commonly asked questions suggested by chatbot before query.

Actors:	User
Trigger:	User opens the FAQs chatbot without entering a query.
Preconditions:	**PRE-1:** The FAQs database must contain commonly asked questions.
Postconditions:	**POST-1:** User is presented with a list of suggested FAQs. **POST-2:** The chatbot learns which FAQs are most frequently accessed.
Normal Flow:	**1.0 View Suggested FAQs** 1. User opens the FAQs chatbot. 2. System displays a list of commonly asked questions. 3. User selects a relevant FAQ. 4. System retrieves and displays the corresponding answer.
Alternative Flows:	**1.1 User Chooses to Enter a Query Instead** 1. User dismisses the suggested FAQs. 2. System allows the user to enter a custom query.
Exceptions:	**EX-1: No frequently asked questions available** 1. System informs the user and provides a manual search option.
Business Rules:	**BR-1:** The chatbot must maintain a list of most-accessed FAQs. **BR-2:** FAQs should be displayed based on popularity and relevance.
Assumptions:	1. Users may prefer browsing FAQs before typing a query.

Table 32 : UC-9.2 View Suggested FAQs

3.3.10 Module 12: Audience Engagement and Communication

UC-10.1 : Receive Notifications for Audience Engagement

In this use case, a creator receives notifications when their content is liked, shared, commented on, or followed. The system ensures updates, allowing creators to engage with their audience effectively.

Actors:	Creator
Trigger:	Audience interacts with the creator's content (like, comment, share, follow).
Preconditions:	**PRE-1:** Creator must have at least one published podcast. **PRE-2:** System must support real-time notifications.
Postconditions:	**POST-1:** Creator is notified of engagement actions. **POST-2:** Notifications are stored in the creator's notification panel.
Normal Flow:	**1.0 Receive Engagement Notifications** 1. Audience interacts with the creator's podcast (like, comment, share, follow). 2. System generates a real-time notification. 3. Creator receives a push or in-app notification. (see EX-1) 4. Creator clicks the notification to view details. 5. System directs the creator to the relevant content.
Alternative Flows:	**1.1 View Notification History** 1. Creator navigates to the "Notifications" section. 2. System displays a history of all past notifications.
Exceptions:	**EX-1: System fails to send real-time notifications** 1. System retries sending notifications. 2. If still unsuccessful, system logs the failure for later delivery.

Business Rules:	**BR-1:** Notifications should be sent in real-time. **BR-2:** Users can configure notification preferences in settings.
Assumptions:	1. Creators actively check their notifications to engage with their audience.

Table 33 : UC-10.1 Receive Notifications for Audience Engagement

UC-10.2 : User-to-User Messaging and Calls

In this use case, users communicate through in-platform messaging or voice/video calls, fostering collaboration and engagement.

.

Actors:	User
Trigger:	User initiates a message or call with another user.
Preconditions:	**PRE-1:** Users must be logged in. **PRE-2:** Messaging and calling features must be enabled.
Postconditions:	**POST-1:** Message or call request is sent and received. **POST-2:** Conversation history is saved.
Normal Flow:	**1.0 Send a Message** 1. User navigates to the messaging section. 2. User searches for another user and selects them. 3. User types a message and clicks "Send". 4. System delivers the message in real time. 5. Recipient receives a notification and responds. **1.1 Start a Voice/Video Call** 1. User selects a contact from the messaging section. 2. User clicks "Call" to initiate voice or video communication. 3. System sends a call request to the recipient. (see 1.1. E1)

	4. If accepted, the call is established. 5. Users communicate and end the call when done.
Alternative Flows:	**1.2 Mute or Block a User** 1. User selects a conversation. 2. User chooses to mute or block the sender. 3. System updates preferences accordingly.
Exceptions:	**EX-1: 1.0. E1: Message fails to send** 1. System retries sending or notifies the user of failure. **EX-2: 1.1. E1: Call request is declined or unanswered** 1. System informs the caller and allows them to retry.
Business Rules:	**BR-1:** Users can only message or call those who have enabled the feature. **BR-2:** Blocked users cannot send messages or call.
Assumptions:	1. Users actively use messaging and calling for engagement.

Table 34 : UC-10.2 User-to-User Messaging and Calls

UC-10.3 : Engage with Podcasts (Like, Comment, Share)

In this use case, a user interacts with podcasts by liking, commenting, or sharing them.

Actors:	User
Trigger:	User selects an engagement option (Like, Comment, Share) on a podcast.

Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The podcast must be publicly available.
Postconditions:	**POST-1:** Engagement action is recorded in the system. **POST-2:** The creator is notified of engagement.
Normal Flow:	**1.0 Engage with a Podcast** 1. User plays a podcast episode. 2. User selects an engagement action: o Like: System records the like. o Comment: User enters a comment and submits. (see EX-1) o Share: User selects a platform to share the podcast.
Alternative Flows:	**1.1 Delete a Comment** 1. User navigates to their comment on a podcast. 2. User selects "Delete". 3. System removes the comment.
Exceptions:	**EX-1: Comment contains restricted content** 1. System flags and removes the comment.
Business Rules:	**BR-1:** Users can engage with public podcasts only. **BR-2:** Comments must adhere to community guidelines.
Assumptions:	1. Users actively engage with content.

Table 35 : UC-10.3 Engage with Podcasts (Like, Comment, Share)

UC-10.4 : Provide User Feedback

In this use case, a user provides feedback at the end of a podcast session. The system prompts the user to rate the session, submit written feedback, and displays a confirmation message. The feedback is then analyzed by the system to identify trends and improve AI responses over time.

Actors:	User, System
Trigger:	System prompts the user for feedback after the podcast session ends.
Preconditions:	**PRE-1:** A podcast session must be completed. **PRE-2:** User must be logged in.
Postconditions:	**POST-1:** User feedback is recorded. **POST-2:** System updates ratings and stores comments. **POST-3:** AI improvements are triggered based on repeated feedback trends.
Normal Flow:	**1.0 Provide User Feedback** 1. System displays a feedback prompt after the session ends. 2. User selects a rating (e.g., 1-5 stars). 3. User optionally enters a written comment on conversation quality, voice clarity, or knowledge accuracy. 4. System stores feedback in the database. 5. System analyzes feedback trends to improve AI performance. 6. System displays a thank-you message.
Alternative Flows:	**1.1 Feedback Skipped** 1. User chooses not to submit feedback. 2. System does not record any data.
Exceptions:	**EX-1: System fails to store feedback** 1. System retries saving or notifies the user of an error.
Business Rules:	**BR-1:** Users can submit feedback only for completed sessions. **BR-2:** Feedback should include both ratings and optional comments. **BR-3:** AI should be continuously refined based on user feedback.

Assumptions:	1. Users will provide constructive feedback to improve AI responses.

Table 36 : UC-10.4 Provide User Feedback

3.4 Event Response Table

3.4.1 ERT-1: User Login

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
1	User logs in	Login credentials (email/password or social login)	User attempts authentication	Invalid credentials (incorrect email/password)	Successfully authenticates and loads user dashboard	Displays an error message prompting the user to try again	User session becomes active, allowing access to the system

3.4.2 ERT-2: Profile Update

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
2	User edits profile info	Name, bio, profile picture, contact details	User submits profile changes	Invalid input format (unsupported characters, empty fields)	Successfully updates the database and reflects changes in the profile	Rejects update and prompts the user to correct input	User profile is successfully updated and displayed

3.4.3 ERT-3: Voice Cloning

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
3	User uploads voice sample	Audio file (WAV, MP3)	User submits a valid sample	Poor audio quality (background noise, distortion)	Processes the sample, generates an AI voice, and saves it for user use	Requests a new upload with clearer audio	AI-generated voice is successfully available for use in applications

3.4.4 ERT-4: Knowledge Base Upload

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
4	User uploads knowledge document	File (PDF, URL, CSV, JSON)	Data processing in progress	File unreadable (corrupted, unsupported format)	Stores and indexes knowledge data, making it searchable	Requests re-upload with a supported format	Knowledge is successfully stored and indexed for future queries

3.4.5 ERT-5: Chatbot Query Processing

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
5	User submits a chatbot query	Query text (user question)	Query processing in progress	No relevant data available	Retrieves the best matching answer and	Informs the user that no relevant answer is available	FAQ response is displayed if a match is found,

					displays it to the user		otherwise user is prompted for clarificati on

3.4.6 ERT-6: Terminate Test Section

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
6	Terminate Section When Time Over	Test section duration (time limit)	Timer for the section expires	N/A	The system automatically ends the section, saves responses, and proceeds to the next section	N/A	The next section starts automatic ally on the screen

3.4.7 ERT-7: AI Host Podcast Management

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
7	AI Host starts podcast	Podcast metadata (title, topic, participants)	AI Host initialized	System failure (crash, unavailable resources)	Successfully introduces and manages the podcast session	Retries starting the podcast session automatically	AI-driven session is in progress and being recorded

3.4.8 ERT-8: AI Guest Response

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
8	AI Guest receives	Query text (spoken	AI processes input	No relevant data	Generates an appropriate AI-driven	Provides an alternative response or	AI Guest successfully responds to

	a question	or typed question)			response with context	asks for clarification	the user with relevant information

3.4.9 ERT-9: Podcast Recording

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
9	User records podcast	Audio stream (live microphone input)	Audio recording active	Microphone issue (hardware failure, permissions denied)	Saves the recorded session successfully	Prompts user to retry or check microphone settings	Podcast recording is ready for editing and post-processing

3.4.10 ERT-10: User Edits Podcast Recording

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
10	User edits podcast recording	Audio segments, edit commands (trim, splice)	Editing session active	Corrupt audio file	Applies edits and updates the podcast recording	Rejects changes and prompts user to re-edit	Edited version saved successfully

3.4.11 ERT-11: User Exports Podcast Audio/Video

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
11	User exports podcast audio/video	Selected file format (MP3,	Export in progress	Format unsupported	Successfully converts and saves file	Prompts user to select a	Podcast ready for publishing

		WAV, MP4)				valid format	

3.4.12 ERT-12: User Subscribes to Premium Plan

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
12	User subscribes to premium plan	Payment details, user account	Subscription activation pending	Payment failure	Activates premium features and updates billing	Notifies user of failed transaction and prompts retry	Subscription activated or remains inactive

3.4.13 ERT-13: Payment Fails

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
13	Payment fails	Transaction details, error message	Payment processing	Invalid card details or insufficient funds	Displays error message and requests retry	Cancels transaction	Transaction incomplete, user remains unsubscribed

3.4.14 ERT-14: User Rates a Podcast Session

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
14	User rates a podcast session	Rating value (1-5 stars), podcast ID	Rating submission pending	Invalid rating value	Stores rating and updates average score	Rejects rating and prompts user to try again	Rating recorded successfully

3.4.15 ERT-15: System Generates Podcast Summary

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
15	System generates podcast summary	Podcast transcript, AI processing	Summary generation in progress	Incomplete transcript	Generates AI-based summary and stores it	Logs error and requests additional processing	Summary available for users

3.4.16 ERT-16: User Interacts with AI-Powered Notifications

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
16	User interacts with AI-powered notifications	Notification content, user engagement data	Notification system active	No relevant notifications	Delivers relevant updates and alerts	No updates available	Notification displayed to the user

4. Functional Requirements

4.1 UC-1 Register Account

4.1.1 FR-1: Account.Register

Identifier	FR-1
Title	Account.Register
Requirement	The system shall allow users to create an account using email and password, verifying email uniqueness and enforcing password strength rules.
Source	Sadia
Rationale	Users must register to access the system securely.
Business Rule	BR-1: Each email must be unique.
Dependencies	FR-2, FR-3
Priority	High

4.1.2 FR-2: Account.Register.SocialLogin

Identifier	FR-2
Title	Account.Register.SocialLogin
Requirement	The system shall allow users to register using Google login, retrieving necessary user details and skipping email verification.

Source	Sadia
Rationale	Provides seamless account creation without requiring additional credentials.
Business Rule	BR-1: Each email linked with social accounts must be unique.
Dependencies	FR-1
Priority	High

4.1.3 FR-3: Account. ValidateEmail

Identifier	FR-3
Title	Account. ValidateEmail
Requirement	When the user enters their email, the system shall verify that it is not already registered.
Source	Sadia
Rationale	To ensure unique user accounts.
Business Rule	BR-1: Each email must be unique.
Dependencies	FR-4
Priority	High

4.1.4 FR-4: Account.EmailVerification

Identifier	FR-4
Title	Account.EmailVerification
Requirement	The system shall send an email verification link to the user and restrict access until the email is verified.
Source	Sadia
Rationale	Ensures security and prevents fraudulent account creation.
Business Rule	BR-1: Users must verify their email before accessing the platform.
Dependencies	FR-3
Priority	High

4.1.5 FR-5: Account.EmailVerification.Expiration

Identifier	FR-5
Title	Account.EmailVerification.Expiration
Requirement	The system shall set a 24-hour expiration period for email verification links and allow users to request a new one if expired.
Source	Sadia
Rationale	Enhances security by ensuring links do not remain valid indefinitely.
Business Rule	N/A

Dependencies	FR-4
Priority	Medium

4.1.6 FR-6: Account.PasswordStrengthValidation

Identifier	FR-6
Title	Account.PasswordStrengthValidation
Requirement	The system shall enforce password strength criteria, requiring at least one uppercase letter, one lowercase letter, one number, and a minimum of eight characters.
Source	Sadia
Rationale	Ensures users create secure passwords to protect their accounts.
Business Rule	N/A
Dependencies	FR-1
Priority	High

4.2 UC-1.2 User Login

4.2.1 FR-7: Account.Login

Identifier	FR-7
Title	Account.Login
Requirement	The system shall allow users to log in using email and password after successful registration and verification.
Source	Rabeea
Rationale	Users must authenticate themselves to access the system.
Business Rule	BR-1: Only verified accounts can log in.
Dependencies	FR-8
Priority	High

4.2.2 FR-8: Account.Authenticate

Identifier	FR-8
Title	Account.Authenticate
Requirement	The system shall validate user credentials against stored records before granting access.
Source	Rabeea

Rationale	Prevents unauthorized access
Business Rule	BR-1: Only verified accounts can log in.
Dependencies	FR-7
Priority	High

4.2.3 FR-9: Account.LoginError

Identifier	FR-6
Title	Account.LoginError
Requirement	If login fails, the system shall display an appropriate error message indicating incorrect credentials or an unverified account.
Source	Rabeea
Rationale	Helps users resolve login issues.
Business Rule	BR-1: Users should have a limited number of login attempts before temporary lockout.
Dependencies	FR-8
Priority	Medium

4.3 UC-1.3 Google Authentication

4.3.1 FR-10: Account.GoogleLogin

Identifier	FR-10
Title	Account.GoogleLogin
Requirement	The system shall allow users to sign in using their Google credentials via OAuth authentication.
Source	Sadia
Rationale	Provides a seamless and secure login experience.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

4.3.2 FR-11: Account.GoogleRedirect

Identifier	FR-11
Title	Account.GoogleRedirect
Requirement	Upon selecting Google login, the system shall redirect the user to Google's authentication page.
Source	Sadia
Rationale	Ensures authentication is done securely through Google's authorization servers.
Business Rule	N/A
Dependencies	FR-10
Priority	High

4.3.3 FR-12: Account.GoogleCredentialVerification

Identifier	FR-12
Title	Account.GoogleCredentialVerification
Requirement	The system shall verify user identity and retrieve necessary user details (name, email, profile picture) from Google.
Source	Sadia
Rationale	Ensures only valid Google accounts can access the system.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

4.3.4 FR-13: Account.GoogleAutoRegister

Identifier	FR-13
Title	Account.GoogleAutoRegister
Requirement	If the user is logging in with Google for the first time, the system shall create a new account using the retrieved Google details.
Source	Sadia
Rationale	Allows new users to register seamlessly without filling out additional forms.
Business Rule	N/A
Dependencies	FR-12
Priority	High

4.3.5 FR-14: Account.GoogleLinkedLogin

Identifier	FR-14
Title	Account.GoogleLinkedLogin
Requirement	If a user has previously linked their Google account, they shall be logged in without re-entering credentials.
Source	Sadia
Rationale	Enhances the login experience by reducing repeated authentication steps.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

4.3.6 FR-15: Account.GoogleAuthFailure

Identifier	FR-15
Title	Account.GoogleAuthFailure
Requirement	If authentication with Google fails, the system shall display an error message and prompt the user to retry or use another login method.
Source	Sadia
Rationale	Ensures users can retry authentication or choose an alternative login method.
Business Rule	BR-1: If authentication fails multiple times, users should be prompted to verify their Google account manually.

Dependencies	FR-11
Priority	Medium

4.4 UC-1.4 Password Recovery

4.4.1 FR-16: Account.PasswordResetRequest

Identifier	FR-16
Title	Account.PasswordResetRequest
Requirement	The system shall allow users to request a password reset link by entering their registered email address.
Source	Rabeea
Rationale	Enables users to regain access to their accounts in case of forgotten passwords.
Business Rule	N/A
Dependencies	FR-17
Priority	High

4.4.2 FR-17: Account.ResetLinkGeneration

Identifier	FR-17
Title	Account.ResetLinkGeneration
Requirement	The system shall generate a one-time password reset link and send it to the user’s registered email address.
Source	Rabeea
Rationale	Ensures secure password recovery.

Business Rule	BR-1: Reset links must be unique and expire after a set time.
Dependencies	FR-16
Priority	High

4.4.3 FR-18: Account.ResetLinkExpiration

Identifier	FR-18
Title	Account.ResetLinkExpiration
Requirement	The system shall set a 30-minute expiration period for password reset links.
Source	Rabeea
Rationale	Prevents unauthorized access through old reset links.
Business Rule	N/A
Dependencies	FR-17
Priority	High

4.4.4 FR-19: Account.InvalidResetRequest

Identifier	FR-19
Title	Account.InvalidResetRequest
Requirement	If the email entered for password recovery is not registered, the system shall display an error message and prompt the user to enter a valid email.
Source	Rabeea
Rationale	Ensures users provide a valid email for password reset.
Business Rule	N/A

Dependencies	FR-16
Priority	Medium

4.4.5 FR-20: Account.NewPasswordValidation

Identifier	FR-20
Title	Account.NewPasswordValidation
Requirement	The system shall enforce password strength requirements for new passwords, ensuring security.
Source	Rabeea
Rationale	Protects accounts from weak password vulnerabilities.
Business Rule	BR-1: The new password must meet a minimum strength requirement.
Dependencies	FR-21
Priority	High

4.4.6 FR-21: Account.PasswordUpdate

Identifier	FR-21
Title	Account.PasswordUpdate
Requirement	Once the user enters a valid new password, the system shall update the password and allow the user to log in.
Source	Rabeea
Rationale	Ensures the user can regain access after password reset.
Business Rule	N/A

Dependencies	FR-20
Priority	High

4.5 UC-1.5 Logout

4.5.1 FR-22: Account.Logout

Identifier	FR-22
Title	Account.Logout
Requirement	The system shall allow users to log out, terminating their active session securely.
Source	Rabeea
Rationale	Ensures session security.
Business Rule	N/A
Dependencies	N/A
Priority	High

4.5.2 FR-23: Account.SessionTermination

Identifier	FR-23
Title	Account.SessionTermination
Requirement	Upon logout, the system shall clear all session-related data and redirect the user to the login page.
Source	Rabeea
Rationale	Prevents unauthorized access to sensitive data.
Business Rule	N/A
Dependencies	FR-22
Priority	High

4.5.3 FR-24: Account.LogoutConfirmation

Identifier	FR-24
Title	Account.LogoutConfirmation
Requirement	The system shall display a confirmation prompt when a user attempts to log out.
Source	Rabeea
Rationale	Prevents accidental logouts.
Business Rule	N/A
Dependencies	FR-22
Priority	Medium

4.6 UC-1.6: Access Sample Podcasts

4.6.1 FR-25: Podcasts.ViewSamples

Identifier	**FR-25**
Title	Podcasts.ViewSamples
Requirement	The system shall allow users to browse and view sample podcasts from the dashboard.
Source	System
Rationale	Users may explore sample podcasts for inspiration before starting their own podcast.
Business Rule	BR-1: Only verified users can access sample podcasts.
Dependencies	None
Priority	High

4.6.2 FR-26: Podcasts.SearchSamples

Identifier	**FR-26**
Title	Podcasts.SearchSamples
Requirement	The system shall allow users to search for specific sample podcasts using filters.
Source	System
Rationale	Users should be able to find specific sample podcasts efficiently.
Business Rule	BR-1: Only verified users can search sample podcasts.
Dependencies	FR-25
Priority	Medium

4.7 UC-1.7: Start a Podcast Session

4.7.1 FR-27: PodcastSession.Start

Identifier	**FR-27**
Title	PodcastSession.Start
Requirement	The system shall allow users to start a new podcast session with either an AI Host or AI Guest.
Source	System
Rationale	Users should be able to initiate a podcast session as part of their creative process.
Business Rule	BR-1: Only verified users can start a podcast session.
Dependencies	None
Priority	High

4.7.2 FR-28: PodcastSession.Resume

Identifier	**FR-28**

Title	PodcastSession.Resume
Requirement	If a session is incomplete, the system shall allow users to resume from the last saved session.
Source	System
Rationale	Users should be able to continue an unfinished podcast session without losing progress.
Business Rule	BR-1: Only users with previous session data can resume.
Dependencies	FR-27
Priority	Medium

4.8 UC-1.8: View Dashboard

4.8.1 FR-29: Dashboard.ViewContent

Identifier	**FR-29**
Title	Dashboard.ViewContent
Requirement	The system shall display a list of relevant podcast content based on the user's profile and followed creators.
Source	System
Rationale	Users should see personalized content upon accessing the dashboard.
Business Rule	BR-1: Content must adhere to moderation policies.
Dependencies	None
Priority	High

4.8.2 FR-30: Dashboard.HandleNoConnection

Identifier	**FR-30**
Title	Dashboard.HandleNoConnection
Requirement	The system shall display an error message and attempt a reload when no internet connection is available.
Source	System
Rationale	Users should be informed of connectivity issues when accessing the dashboard.
Business Rule	None
Dependencies	FR-29
Priority	Medium

4.9 UC-1.9: Create and Personalize User Profile

4.9.1 FR-31: Profile.Create

Identifier	**FR-31**
Title	Profile.Create
Requirement	The system shall allow users to create and personalize their profile by adding a display name, bio, and profile picture.
Source	System
Rationale	A personalized profile helps users engage with the platform.
Business Rule	BR-1: Users must provide a unique display name.
Dependencies	None
Priority	High

4.9.2 FR-32: Profile.Update

Identifier	**FR-32**
Title	Profile.Update
Requirement	Users shall be able to update their profile details, including bio and profile picture.
Source	System
Rationale	Users should have the flexibility to update their profile information.
Business Rule	BR-1: Profile updates should be saved immediately.
Dependencies	FR-31
Priority	High

4.10 UC-1.10: Edit Profile

4.10.1 FR-33: Profile.EditDetails

Identifier	**FR-33**
Title	Profile.EditDetails
Requirement	Users shall be able to edit their username, phone number, and password.
Source	System
Rationale	Users need to manage their profile information efficiently.
Business Rule	BR-1: Only one account can be registered per username.
Dependencies	FR-31, FR-32
Priority	High

4.10.2 FR-34: Profile.ValidatePassword

Identifier	**FR-34**

Title	Profile.ValidatePassword
Requirement	The system shall validate that new passwords meet the required security criteria.
Source	System
Rationale	Ensuring strong passwords enhances security.
Business Rule	BR-1: Password must contain alphabets and digits.
Dependencies	FR-33
Priority	High

4.11 UC-1.11: Follow Creators and View Updates

4.11.1 FR-35: FollowCreator

Identifier	**FR-35**
Title	FollowCreator
Requirement	The system shall allow users to follow and unfollow creators.
Source	System
Rationale	Users should be able to curate their content feed.
Business Rule	BR-1: Users can follow or unfollow creators at any time.
Dependencies	None
Priority	High

4.11.2 FR-36: ViewCreatorUpdates

Identifier	**FR-36**

Title	ViewCreatorUpdates
Requirement	The system shall display updates from followed creators in the user’s feed.
Source	System
Rationale	Users should be informed of new content from creators they follow.
Business Rule	BR-1: Feed updates should be in real-time.
Dependencies	FR-35
Priority	High

4.12 UC-1.12: View Activity Feed of Followed Creators

4.12.1 FR-37: ActivityFeed.View

Identifier	**FR-37**
Title	ActivityFeed.View
Requirement	The system shall allow users to view recent activities of followed creators.
Source	System
Rationale	Users should stay updated on creator activities.
Business Rule	BR-1: Activity feed updates should be in real-time.
Dependencies	FR-36
Priority	High

4.12.2 FR-38: ActivityFeed.HandleNoData

Identifier	**FR-38**

Title	ActivityFeed.HandleNoData
Requirement	If no activity is available, the system shall inform the user and suggest following more creators.
Source	System
Rationale	Users should understand why their activity feed is empty.
Business Rule	None
Dependencies	FR-37
Priority	Medium

4.13 UC-2.1 Voice Cloning Setup

4.13.1 FR-39: VoiceCloning.RecordSample

Identifier	**FR-39**
Title	VoiceCloning.RecordSample
Requirement	The system shall allow users to record a voice sample for cloning.
Source	Rabeea
Rationale	Enables AI-generated voices.
Business Rule	BR-11: Recording must be clear and meet quality standards.
Dependencies	FR-40
Priority	High

4.13.2 4.13.2 FR-40: VoiceCloning.UploadSample

Identifier	**FR-40**
Title	VoiceCloning.UploadSample

Requirement	The system shall allow users to upload a voice sample in supported formats.
Source	Rabeea
Rationale	Provides flexibility for AI voice generation.
Business Rule	BR-12: Only WAV, MP3 formats are allowed.
Dependencies	FR-41
Priority	High

4.13.3 FR-41: VoiceCloning.ProcessSample

Identifier	**FR-41**
Title	VoiceCloning.ProcessSample
Requirement	The system shall process the uploaded or recorded voice sample for AI cloning.
Source	Sadia
Rationale	Ensures high-quality AI-generated voice.
Business Rule	BR-13: Processing must maintain original voice characteristics.
Dependencies	FR-42
Priority	High

4.13.4 FR-42: VoiceCloning.PreviewGeneratedVoice

Identifier	**FR-42**
Title	VoiceCloning.PreviewGeneratedVoice
Requirement	The system shall allow users to preview the generated AI voice before saving.
Source	Sadia
Rationale	Allows user verification of quality.

Business Rule	BR-14: Users can re-record or modify AI voice if unsatisfied.
Dependencies	FR-41
Priority	Medium

4.14 UC-3.1 Knowledge Base Setup

4.14.1 FR-43: KnowledgeBase.UploadData

Identifier	**FR-43**
Title	KnowledgeBase.UploadData
Requirement	The system shall allow users to upload structured and unstructured data (PDF, CSV, JSON, URLs).
Source	Rabeea
Rationale	Ensures AI has relevant information for retrieval.
Business Rule	BR-15: Only supported formats can be uploaded.
Dependencies	FR-44
Priority	High

4.14.2 FR-44: KnowledgeBase.ProcessData

Identifier	**FR-44**
Title	KnowledgeBase.ProcessData
Requirement	The system shall clean, format, and structure uploaded data for efficient AI retrieval.
Source	Sadia
Rationale	Ensures high-quality and relevant AI responses.

Business Rule	BR-16: Metadata must be attached to each data chunk.
Dependencies	FR-45
Priority	High

4.14.3 FR-45: KnowledgeBase.VectorEmbedding

Identifier	**FR-45**
Title	KnowledgeBase.VectorEmbedding
Requirement	The system shall generate vector embeddings for better AI retrieval performance.
Source	Rabeea
Rationale	Enhances AI’s ability to match queries with relevant data.
Business Rule	BR-17: Vectorization must ensure minimal data loss.
Dependencies	FR-44
Priority	High

4.14.4 FR-46: KnowledgeBase.MetadataTagging

Identifier	**FR-46**
Title	KnowledgeBase.MetadataTagging
Requirement	The system shall attach metadata to uploaded data for indexing and retrieval.
Source	Sadia
Rationale	Ensures accurate filtering and searching of knowledge base information.
Business Rule	BR-18: Metadata must follow a standard format.
Dependencies	FR-41

Priority	Medium

4.15 UC-4.1 AI Agent Customization

4.15.1 FR-47: AIAgent.CustomizeAppearance

Identifier	**FR-47**
Title	AIAgent.CustomizeAppearance
Requirement	The system shall allow users to set a custom name, avatar, and voice for their AI agent.
Source	Rabeea
Rationale	Enables personalized podcasting experiences.
Business Rule	BR-19: Users must configure AI before recording.
Dependencies	FR-48
Priority	High

4.15.2 FR-48: AIAgent.SavePreferences

Identifier	**FR-48**
Title	AIAgent.SavePreferences
Requirement	The system shall save AI agent settings for future sessions.
Source	Sadia
Rationale	Provides users with a consistent podcasting experience.
Business Rule	BR-20: Users can modify settings before each session.
Dependencies	FR-47
Priority	Medium

4.15.3 FR-49: AIAgent.LoadPreferences

Identifier	**FR-49**
Title	AIAgent.LoadPreferences
Requirement	The system shall allow users to load previously saved AI preferences automatically.
Source	Rabeea
Rationale	Ensures user convenience and efficiency.
Business Rule	BR-21: Preferences must be stored securely.
Dependencies	FR-48
Priority	Medium

4.16 UC-5.1 Record Podcast Session

4.16.1 FR-50: PodcastSession.StartRecording

Identifier	**FR-50**
Title	PodcastSession.StartRecording
Requirement	The system shall allow users to start recording a podcast session with AI-generated or real co-hosts.
Source	Rabeea
Rationale	Enables users to create and capture podcast content.
Business Rule	BR-22: Recording must start only after user confirmation.
Dependencies	FR-51
Priority	High

4.16.2 FR-51: PodcastSession.PauseRecording

Identifier	**FR-51**
Title	PodcastSession.PauseRecording
Requirement	The system shall allow users to pause an ongoing podcast recording and resume later.
Source	Sadia
Rationale	Provides flexibility for users during recordings.
Business Rule	BR-23: Paused recordings should not be lost.
Dependencies	FR-50
Priority	Medium

4.16.3 FR-52: PodcastSession.StopRecording

Identifier	**FR-52**
Title	PodcastSession.StopRecording
Requirement	The system shall allow users to stop a podcast recording and save the session.
Source	Rabeea
Rationale	Ensures completed recordings are stored securely.
Business Rule	BR-24: Users must confirm before stopping the session.
Dependencies	FR-50
Priority	High

4.16.4 FR-53: PodcastSession.SetRecordingLimit

Identifier	**FR-53**
Title	PodcastSession.SetRecordingLimit

Requirement	The system shall enforce a maximum recording time per session.
Source	Rabeea
Rationale	Prevents excessively long recordings that could impact performance.
Business Rule	BR-25: Recording limit should be configurable by the user.
Dependencies	FR-50
Priority	Medium

4.16.5 FR-54: PodcastSession.AutoSaveRecording

Identifier	**FR-54**
Title	PodcastSession.AutoSaveRecording
Requirement	The system shall periodically auto-save the recording to prevent data loss.
Source	Sadia
Rationale	Ensures users do not lose content in case of crashes.
Business Rule	BR-26: Auto-save should trigger every 5 minutes.
Dependencies	FR-50
Priority	High

4.17 UC-5.2 Edit Podcast Recording

4.17.1 FR-56: PodcastSession.LoadEditingInterface

Identifier	**FR-56**
Title	PodcastSession.LoadEditingInterface
Requirement	The system shall load an audio editing interface when the user selects "Edit Recording."

Source	Rabeea
Rationale	Provides users with a dedicated editing workspace.
Business Rule	BR-28: Editing interface should load within 5 seconds.
Dependencies	FR-52
Priority	High

4.17.2 FR-57: PodcastSession.TrimAudio

Identifier	**FR-57**
Title	PodcastSession.TrimAudio
Requirement	The system shall allow users to trim recorded audio to remove unwanted parts.
Source	Sadia
Rationale	Ensures refined podcast content.
Business Rule	BR-29: Trimmed sections should be reversible until final save.
Dependencies	FR-56
Priority	High

4.17.3 FR-58: PodcastSession.RearrangeClips

Identifier	**FR-58**
Title	PodcastSession.RearrangeClips
Requirement	The system shall allow users to rearrange audio clips within the timeline.
Source	Rabeea
Rationale	Provides flexibility in podcast structuring.
Business Rule	BR-30: Rearranged clips must maintain smooth transitions.

Dependencies	FR-56
Priority	High

4.17.4 FR-59: PodcastSession.MergeAudio

Identifier	**FR-59**
Title	PodcastSession.MergeAudio
Requirement	The system shall allow users to merge multiple audio segments into a single file.
Source	Sadia
Rationale	Provides flexibility in podcast editing.
Business Rule	BR-31: Merged audio must maintain original quality.
Dependencies	FR-57, FR-58
Priority	Medium

4.17.5 FR-60: PodcastSession.ApplyFilters

Identifier	**FR-60**
Title	PodcastSession.ApplyFilters
Requirement	The system shall allow users to apply noise reduction and enhancement filters.
Source	Rabeea
Rationale	Ensures high-quality audio output.
Business Rule	BR-32: Audio enhancements must be previewed before final save.
Dependencies	FR-59
Priority	Medium

4.17.6 FR-61: PodcastSession.SaveEditedRecording

Identifier	**FR-61**
Title	PodcastSession.SaveEditedRecording
Requirement	The system shall allow users to save the edited recording.
Source	Sadia
Rationale	Ensures changes are stored securely.
Business Rule	BR-33: Users must confirm before overwriting the original file.
Dependencies	FR-60
Priority	High

4.17.7 FR-62: PodcastSession.UpdateFinalAudio

Identifier	**FR-62**
Title	PodcastSession.UpdateFinalAudio
Requirement	The system shall replace the original recording with the updated final version.
Source	Rabeea
Rationale	Ensures the final version is stored and ready for publishing.
Business Rule	BR-34: System should maintain a backup of the original file.
Dependencies	FR-61
Priority	High

4.18 UC-5.3 Podcast Review Before Saving

4.18.1 FR-63: PodcastSession.PreviewFinalAudio

Identifier	**FR-63**
Title	PodcastSession.PreviewFinalAudio
Requirement	The system shall allow users to preview the final edited podcast before saving.
Source	Rabeea
Rationale	Ensures users can review their content before finalizing.
Business Rule	BR-31: Users must confirm before saving the final version.
Dependencies	FR-62
Priority	High

4.18.2 FR-64: PodcastSession.AdjustVolumeLevels

Identifier	**FR-64**
Title	PodcastSession.AdjustVolumeLevels
Requirement	The system shall allow users to adjust volume levels for consistency before saving.
Source	Sadia
Rationale	Ensures clear and balanced audio output.
Business Rule	BR-32: Volume normalization should be automatic but adjustable.
Dependencies	FR-63
Priority	Medium

4.18.3 FR-65: PodcastSession.AddEpisodeMetadata

Identifier	**FR-65**

Title	PodcastSession.AddEpisodeMetadata
Requirement	The system shall allow users to enter episode metadata (title, description, tags) before saving.
Source	Rabeea
Rationale	Ensures podcasts are properly categorized for future reference.
Business Rule	BR-33: Metadata fields should be required before final save.
Dependencies	FR-64
Priority	High

4.19 UC-5.4 Export and Save Podcast

4.19.1 FR-66: PodcastSession.PublishPodcast

Identifier	**FR-66**
Title	PodcastSession.PublishPodcast
Requirement	The system shall allow users to publish the podcast directly to selected platforms.
Source	Sadia
Rationale	Enables easy distribution of podcasts.
Business Rule	BR-34: Users must authenticate platform access before publishing.
Dependencies	FR-65
Priority	High

4.19.2 FR-67: PodcastSession.GenerateFileName

Identifier	**FR-67**
Title	PodcastSession.GenerateFileName

Requirement	The system shall automatically generate a file name based on episode metadata.
Source	Rabeea
Rationale	Ensures a standardized file naming format.
Business Rule	BR-35: Users can override the generated file name if needed.
Dependencies	FR-65
Priority	Medium

4.19.3 FR-68: PodcastSession.ExportAudio

Identifier	**FR-68**
Title	PodcastSession.ExportAudio
Requirement	The system shall allow users to export the finalized podcast in MP3/WAV formats.
Source	Sadia
Rationale	Enables users to distribute their podcast on different platforms.
Business Rule	BR-36: Users must select a compatible format before export.
Dependencies	FR-67
Priority	High

4.19.4 FR-69: PodcastSession.SaveDraft

Identifier	**FR-69**
Title	PodcastSession.SaveDraft
Requirement	The system shall allow users to save an incomplete podcast session as a draft.
Source	Rabeea
Rationale	Allows users to return and finalize content later.

Business Rule	BR-37: Drafts must be auto-saved periodically.
Dependencies	FR-65
Priority	Medium

4.20 UC-6.1 AI Host & AI Guest Interaction

4.20.1 FR-70: PodcastAI.InitializeSession

Identifier	**FR-70**
Title	PodcastAI.InitializeSession
Requirement	The system shall allow users to initiate a podcast session with an AI Host or AI Guest.
Source	Rabeea
Rationale	Enables AI-powered podcasting experiences.
Business Rule	BR-41: AI settings must be pre-configured before the session starts.
Dependencies	None
Priority	High

4.20.2 FR-71: PodcastAI.LoadSettings

Identifier	**FR-71**
Title	PodcastAI.LoadSettings
Requirement	The system shall load the pre-configured AI settings, voice model, and knowledge base before starting the session.
Source	Sadia
Rationale	Ensures AI uses the correct configurations for smooth interaction.

Business Rule	BR-42: AI must have access to the selected knowledge base before interaction.
Dependencies	FR-70
Priority	High

4.20.3 FR-72: PodcastAI.HostIntroduction

Identifier	**FR-72**
Title	PodcastAI.HostIntroduction
Requirement	The AI Host shall introduce the episode and set the discussion agenda.
Source	Rabeea
Rationale	Provides a structured and engaging podcast experience.
Business Rule	BR-43: AI introductions must align with the selected episode theme.
Dependencies	FR-71
Priority	High

4.20.4 FR-73: PodcastAI.GuestResponses

Identifier	**FR-73**
Title	PodcastAI.GuestResponses
Requirement	The AI Guest shall dynamically respond to user queries and share insights.
Source	Sadia
Rationale	Ensures an interactive and engaging podcast conversation.
Business Rule	BR-44: AI responses must be contextually relevant.
Dependencies	FR-72
Priority	High

4.20.5 FR-74: PodcastAI.ManageTransitions

Identifier	**FR-74**
Title	PodcastAI.ManageTransitions
Requirement	The AI Host shall facilitate smooth topic transitions during the podcast session.
Source	Rabeea
Rationale	Ensures structured and seamless conversations.
Business Rule	BR-45: Transitions should align with the episode flow and pacing.
Dependencies	FR-73
Priority	Medium

4.20.6 FR-75: PodcastAI.GenerateSummary

Identifier	**FR-75**
Title	PodcastAI.GenerateSummary
Requirement	The AI Host shall generate and provide a structured summary of the discussion at the end of the session.
Source	Sadia
Rationale	Allows users to review key discussion points.
Business Rule	BR-46: Summaries must highlight the main topics and insights covered.
Dependencies	FR-74
Priority	Medium

4.21 UC-7.1 Select Avatars for Speakers

4.21.1 FR-76: PodcastAvatars.DisplayPredefinedAvatars

Identifier	**FR-76**
Title	PodcastAvatars.DisplayPredefinedAvatars
Requirement	The system shall provide a selection of predefined avatars for users to choose from.
Source	System
Rationale	Ensures that users have quick access to avatars without uploading their own.
Business Rule	BR-47: Predefined avatars must be diverse and customizable.
Dependencies	None
Priority	High

4.21.2 FR-77: PodcastAvatars.UploadCustomAvatar

Identifier	**FR-77**
Title	PodcastAvatars.UploadCustomAvatar
Requirement	The system shall allow users to upload a custom avatar for each speaker.
Source	Creator
Rationale	Provides personalization options for users.
Business Rule	BR-48: Uploaded avatars must meet file format and size requirements.
Dependencies	FR-76
Priority	High

4.21.3 FR-78: PodcastAvatars.AssignAvatarsToSpeakers

Identifier	**FR-78**

Title	PodcastAvatars.AssignAvatarsToSpeakers
Requirement	The system shall allow users to assign an avatar to each speaker in the podcast.
Source	Creator
Rationale	Ensures each speaker has a unique visual identity.
Business Rule	BR-49: Each speaker must be assigned an avatar before proceeding.
Dependencies	FR-76, FR-77
Priority	High

4.21.4 FR-79: PodcastAvatars.ModifyAvatars

Identifier	**FR-79**
Title	PodcastAvatars.ModifyAvatars
Requirement	The system shall allow users to change assigned avatars before finalizing selections.
Source	Creator
Rationale	Provides flexibility in avatar selection.
Business Rule	BR-50: Users must confirm changes before applying.
Dependencies	FR-78
Priority	Medium

4.21.5 FR-80: PodcastAvatars.ValidateAvatarSelection

Identifier	**FR-80**
Title	PodcastAvatars.ValidateAvatarSelection
Requirement	The system shall prompt users to choose or upload an avatar before proceeding.
Source	System

Rationale	Ensures compliance with avatar selection rules.
Business Rule	BR-51: Users cannot proceed without assigning an avatar.
Dependencies	FR-78
Priority	High

4.22 UC-7.2 Select Visual Elements for Video Podcast

4.22.1 FR-81: PodcastVisuals.DisplayPredefinedVisuals

Identifier	**FR-81**
Title	PodcastVisuals.DisplayPredefinedVisuals
Requirement	The system shall provide a library of background themes, overlays, and effects.
Source	System
Rationale	Allows users to select professional and engaging visuals.
Business Rule	BR-52: Predefined visuals must be categorized for easier selection.
Dependencies	None
Priority	High

4.22.2 FR-82: PodcastVisuals.UploadCustomVisuals

Identifier	**FR-82**
Title	PodcastVisuals.UploadCustomVisuals
Requirement	The system shall allow users to upload custom visuals for the podcast.
Source	Creator
Rationale	Provides customization for unique podcast branding.

Business Rule	BR-53: Uploaded visuals must meet format and resolution guidelines.
Dependencies	FR-81
Priority	High

4.23 UC-7.3 Preview and Export Video Podcast

4.23.1 FR-83: PodcastPreview.PlayVideo

Identifier	**FR-83**
Title	PodcastPreview.PlayVideo
Requirement	The system shall allow users to preview the full video podcast before exporting.
Source	Creator
Rationale	Ensures users can review the final product before publishing.
Business Rule	BR-54: Preview must match the final exported video quality.
Dependencies	None
Priority	High

4.23.2 FR-84: PodcastPreview.ConfirmQuality

Identifier	**FR-84**
Title	PodcastPreview.ConfirmQuality
Requirement	The system shall allow users to confirm video quality before proceeding to export.
Source	Creator
Rationale	Prevents low-quality exports and ensures professional output.
Business Rule	BR-55: Users must approve quality before export.

Dependencies	FR-83
Priority	Medium

4.23.3 FR-85: PodcastExport.ExportVideo

Identifier	**FR-85**
Title	PodcastExport.ExportVideo
Requirement	The system shall allow users to export the video podcast in MP4 format.
Source	Creator
Rationale	Enables easy sharing and publishing.
Business Rule	BR-56: Exported videos must maintain the original quality.
Dependencies	FR-84
Priority	High

4.24 UC-7.4 Request Podcast Summary

4.24.1 FR-86: PodcastSummary.GenerateSummary

Identifier	**FR-86**
Title	PodcastSummary.GenerateSummary
Requirement	The system shall generate an AI-based summary for a selected podcast episode.
Source	System
Rationale	Provides quick insights and key takeaways for users.
Business Rule	BR-57: Summaries must highlight main discussion points.
Dependencies	None

Priority	High

4.24.2 FR-87: PodcastSummary.CustomizeSummaryFormat

Identifier	**FR-87**
Title	PodcastSummary.CustomizeSummaryFormat
Requirement	The system shall allow users to choose between a detailed or brief summary format.
Source	User
Rationale	Provides flexibility in how users consume summaries.
Business Rule	BR-58: Users must be able to switch summary formats dynamically.
Dependencies	FR-86
Priority	Medium

4.25 UC-7.5 Search for Specific Information in a Podcast Summary

4.25.1 FR-88: PodcastSummary.SearchKeywords

Identifier	**FR-88**
Title	PodcastSummary.SearchKeywords
Requirement	The system shall allow users to search for specific topics or keywords within a podcast summary.
Source	User
Rationale	Helps users find relevant content quickly.
Business Rule	BR-59: Search results must be contextually relevant.
Dependencies	FR-86
Priority	High

4.25.2 FR-89: PodcastSummary.HighlightResults

Identifier	**FR-89**
Title	PodcastSummary.HighlightResults
Requirement	The system shall highlight matching sections of the summary based on the search query.
Source	System
Rationale	Improves user experience in locating information.
Business Rule	BR-60: Highlighted sections must be clearly visible.
Dependencies	FR-88
Priority	Medium

4.26 UC-7.6 Rate Podcast Session

4.26.1 FR-90: PodcastRating.ProvideRating

Identifier	**FR-90**
Title	PodcastRating.ProvideRating
Requirement	The system shall allow users to rate a completed podcast session on a scale (e.g., 1-5 stars).
Source	User
Rationale	Collects user feedback for improving podcast quality.
Business Rule	BR-61: Users can only rate sessions they have hosted or attended.
Dependencies	None
Priority	High

4.26.2 FR-91: PodcastRating.PreventDuplicateRatings

Identifier	**FR-91**
Title	PodcastRating.PreventDuplicateRatings
Requirement	The system shall prevent users from rating the same session multiple times.
Source	System
Rationale	Ensures fair and accurate rating data.
Business Rule	BR-62: Users can update their rating but not submit duplicates.
Dependencies	FR-90
Priority	Medium

4.27 UC-8.1 Subscription Management

4.27.1 FR-92: Subscription.DisplayPlans

Identifier	**FR-92**
Title	Subscription.DisplayPlans
Requirement	The system shall display all available subscription plans.
Source	System
Rationale	Ensures users can view and compare plans before subscribing.
Business Rule	BR-63: Plans must include pricing, features, and billing cycle.
Dependencies	None
Priority	High

4.27.2 FR-93: Subscription.ProcessPayment

Identifier	**FR-93**
Title	Subscription.ProcessPayment
Requirement	The system shall securely process subscription payments.
Source	System
Rationale	Ensures safe transactions for subscription-based access.
Business Rule	BR-64: Payment must be validated through a secure gateway.
Dependencies	FR-92
Priority	High

4.27.3 FR-94: Subscription.ManageAutoRenewal

Identifier	**FR-94**
Title	Subscription.ManageAutoRenewal
Requirement	The system shall allow users to enable or disable auto-renewal.
Source	User
Rationale	Provides users control over their recurring payments.
Business Rule	BR-65: Auto-renewal settings must be user-configurable.
Dependencies	FR-93
Priority	Medium

4.27.4 FR-95: Subscription.HandlePaymentFailure

Identifier	**FR-95**
Title	Subscription.HandlePaymentFailure

Requirement	The system shall notify users and prompt them to update payment details if payment fails.
Source	System
Rationale	Prevents unintended subscription cancellations.
Business Rule	BR-66: Users must be given at least one retry attempt.
Dependencies	FR-93
Priority	High

4.28 UC-8.2 Free Content Access Limit

4.28.1 FR-96: FreeAccess.TrackUsage

Identifier	**FR-96**
Title	FreeAccess.TrackUsage
Requirement	The system shall track the number of free content sessions used by a user.
Source	System
Rationale	Ensures users do not exceed free content limits.
Business Rule	BR-67: Free usage count must reset after a billing cycle.
Dependencies	None
Priority	High

4.28.2 FR-97: FreeAccess.PromptSubscription

Identifier	**FR-97**
Title	FreeAccess.PromptSubscription
Requirement	The system shall prompt users to subscribe once the free content limit is reached.

Source	System
Rationale	Encourages users to upgrade for unlimited access.
Business Rule	BR-68: Prompt must include subscription benefits.
Dependencies	FR-96
Priority	High

4.29 UC-8.3 Subscription Expiry Reminder

4.29.1 FR-98: Subscription.SendExpiryNotification

Identifier	**FR-98**
Title	Subscription.SendExpiryNotification
Requirement	The system shall notify users before their subscription expires.
Source	System
Rationale	Ensures users are aware of expiration dates to prevent service interruption.
Business Rule	BR-69: Notifications must be sent at least 3 days before expiry.
Dependencies	FR-94
Priority	High

4.29.2 FR-99: Subscription.RenewSubscription

Identifier	**FR-99**
Title	Subscription.RenewSubscription
Requirement	The system shall allow users to renew their subscription before expiry.
Source	User

Rationale	Prevents loss of access due to expired subscriptions.
Business Rule	BR-70: Renewals must process through a secure payment gateway.
Dependencies	FR-98
Priority	High

4.30 UC-9.1 Ask a Question from FAQs Chatbot

4.30.1 FR-100: FAQsChatbot.OpenChatbot

Identifier	**FR-100**
Title	FAQsChatbot.OpenChatbot
Requirement	The system shall allow users to open the FAQs chatbot from the menu.
Source	System
Rationale	Ensures users can quickly access FAQ support.
Business Rule	BR-71: The chatbot must be accessible at all times.
Dependencies	None
Priority	High

4.30.2 FR-101: FAQsChatbot.ProcessUserQuery

Identifier	**FR-101**
Title	FAQsChatbot.ProcessUserQuery
Requirement	The system shall process user queries and retrieve relevant FAQ answers.
Source	System

Rationale	Ensures that users get accurate responses to their questions.
Business Rule	BR-72: Responses must come from the verified FAQs database.
Dependencies	FR-100
Priority	High

4.30.3 FR-102: FAQsChatbot.HandleNoMatch

Identifier	**FR-102**
Title	FAQsChatbot.HandleNoMatch
Requirement	If no exact match is found, the system shall suggest related FAQs.
Source	System
Rationale	Provides alternative answers when a direct match is unavailable.
Business Rule	BR-73: Suggested FAQs must be contextually relevant.
Dependencies	FR-101
Priority	Medium

4.30.4 FR-103: FAQsChatbot.ImproveResponses

Identifier	**FR-103**
Title	FAQsChatbot.ImproveResponses
Requirement	The system shall log chatbot interactions to improve future responses.
Source	System
Rationale	Helps refine chatbot accuracy over time.
Business Rule	BR-74: User interactions should be analyzed periodically.
Dependencies	FR-101

Priority	Medium

4.31 UC-9.2 View Suggested FAQs

4.31.1 FR-104: FAQsChatbot.DisplayCommonQuestions

Identifier	**FR-104**
Title	FAQsChatbot.DisplayCommonQuestions
Requirement	The system shall display commonly asked questions when the chatbot is opened.
Source	System
Rationale	Helps users find answers quickly without typing queries.
Business Rule	BR-75: The FAQ list should update based on recent user activity.
Dependencies	None
Priority	High

4.31.2 FR-105: FAQsChatbot.AllowFAQSelection

Identifier	**FR-105**
Title	FAQsChatbot.AllowFAQSelection
Requirement	The system shall allow users to select a suggested FAQ to view its answer.
Source	User
Rationale	Simplifies navigation within the chatbot.
Business Rule	BR-76: Selected FAQs must display full, structured responses.
Dependencies	FR-104
Priority	High

4.31.3 FR-106: FAQsChatbot.HandleNoFAQsAvailable

Identifier	**FR-106**
Title	FAQsChatbot.HandleNoFAQsAvailable
Requirement	If no suggested FAQs are available, the system shall provide a manual search option.
Source	System
Rationale	Ensures users can still find answers even if no common FAQs exist.
Business Rule	BR-77: Search results must prioritize relevance.
Dependencies	FR-104
Priority	Medium

4.32 UC-10.1 Receive Notifications for Audience Engagement

4.32.1 FR-107: Notifications.GenerateEngagementNotification

Identifier	**FR-107**
Title	Notifications.GenerateEngagementNotification
Requirement	The system shall generate real-time notifications for likes, comments, shares, and follows.
Source	System
Rationale	Ensures creators are informed of audience interactions.
Business Rule	BR-78: Notifications must be sent instantly.
Dependencies	None
Priority	High

4.32.2 FR-108: Notifications.StoreEngagementHistory

Identifier	**FR-108**
Title	Notifications.StoreEngagementHistory
Requirement	The system shall store all engagement notifications in the creator’s notification panel.
Source	System
Rationale	Allows creators to review past interactions.
Business Rule	BR-79: Notifications should be stored for at least 30 days.
Dependencies	FR-107
Priority	Medium

4.33 UC-10.2 User-to-User Messaging and Calls

4.33.1 FR-109: Messaging.SendMessage

Identifier	**FR-109**
Title	Messaging.SendMessage
Requirement	The system shall allow users to send real-time text messages.
Source	User
Rationale	Enables seamless communication between users.
Business Rule	BR-80: Users can only message those who have enabled messaging.
Dependencies	None
Priority	High

4.33.2 FR-110: Messaging.InitiateCall

Identifier	**FR-110**
Title	Messaging.InitiateCall
Requirement	The system shall allow users to initiate voice and video calls.
Source	User
Rationale	Supports real-time collaboration.
Business Rule	BR-81: Calls can only be made to users with calling enabled.
Dependencies	None
Priority	High

4.34 UC-10.3 Engage with Podcasts (Like, Comment, Share)

4.34.1 FR-111: PodcastEngagement.LikePodcast

Identifier	**FR-111**
Title	PodcastEngagement.LikePodcast
Requirement	The system shall allow users to like podcast episodes.
Source	User
Rationale	Encourages audience interaction.
Business Rule	BR-82: Users can like a podcast only once.
Dependencies	None
Priority	High

4.34.2 FR-112: PodcastEngagement.CommentOnPodcast

Identifier	**FR-112**
Title	PodcastEngagement.CommentOnPodcast
Requirement	The system shall allow users to comment on podcast episodes.
Source	User
Rationale	Facilitates discussion and feedback.
Business Rule	BR-83: Comments must follow community guidelines.
Dependencies	None
Priority	High

4.35 UC-10.4 Provide User Feedback

4.35.1 FR-113: Feedback.CollectPodcastRating

Identifier	**FR-113**
Title	Feedback.CollectPodcastRating
Requirement	The system shall prompt users to rate a podcast session after it ends.
Source	System
Rationale	Helps gather audience feedback.
Business Rule	BR-84: Ratings should be between 1-5 stars.
Dependencies	None
Priority	High

4.35.2 FR-114: Feedback.SubmitWrittenFeedback

Identifier	**FR-114**
Title	Feedback.SubmitWrittenFeedback
Requirement	The system shall allow users to submit written feedback about a podcast session.
Source	User
Rationale	Enables users to provide detailed input for improvement.
Business Rule	BR-85: Feedback should be stored for AI analysis.
Dependencies	FR-113
Priority	Medium

5. Non-Functional Requirements

This section specifies non-functional requirements other than constraints. These quality requirements should be specific, quantifiable and verifiable. The non-functional requirements related to our system includes Usability, Performance, Security, Interoperability, and Integrity.

5.1 Usability

USE-1: 95% of users shall be able to navigate and use PodVerse within 5 minutes of initial interaction.
USE-2: The user interface shall be intuitive and accessible, ensuring that users can easily manage podcasts, interact with AI features, and explore content.
USE-3: The AI-powered features (e.g., AI Host, AI Guest, Podcast Summary) shall correctly interpret and respond to at least 90% of user queries.
USE-4: Users shall be able to initiate and maintain conversations, subscriptions, and interactions with minimal effort and latency.
USE-5: Error messages and system alerts shall be clear, concise, and user-friendly to guide users in case of failures or incorrect inputs.

5.2 Perfomance

PER-1: AI-generated podcast summaries shall be generated within 10 seconds of a user request.
PER-2: The system shall start podcast recording within 3 seconds of user initiation.
PER-3: The system shall retrieve and load stored podcasts within 5 seconds of user request.

PER-4: Live interactions between AI and users in a podcast session shall maintain a response latency of less than 3 seconds.

5.3 Security

SEC-1: All passwords shall be stored in encrypted form.
SEC-2: Only authenticated users shall be allowed to create, edit, or publish podcasts.
SEC-3: User interactions, feedback, and messages shall be securely stored, ensuring privacy and data integrity.
SEC-4: The system shall implement role-based access control, restricting creator and viewer privileges appropriately.
SEC-5: All payment transactions shall be securely processed using encrypted connections.

5.4 Interoperability

IOP-1: PodVerse shall support audio file uploads in MP3 and WAV formats for podcast recording and voice cloning.
IOP-2: The system shall integrate with third-party authentication services such as Google for login.
IOP-3: PodVerse shall support video podcast exports in MP4 format.

5.5 Integrity

INT-1: The system shall protect against unauthorized modification, deletion, or corruption of podcast content.
INT-2: Uploaded podcast recordings shall remain unaltered and retrievable in their original form unless edited by the user.
INT-3: PodVerse shall support video podcast exports in MP4 format.

6. External Interface Requirements

6.1 User Interfaces Requirements

UI-1: PodVerse shall provide a light mode-friendly UI for a comfortable viewing experience.
UI-2: A left panel shall allow quick access to Dashboard, Podcasts, Messages, and Profile.
UI-3: A search bar shall be prominently displayed on the main dashboard for users to quickly find podcasts, creators, and topics.
UI-4: The system shall provide real-time error messages and user-friendly notifications to guide users in resolving issues efficiently.

6.2 Software interfaces

SI-1: PodVerse shall integrate with Google OAuth API for secure user authentication via Google login.

SI-2: The system shall support MP3, WAV, and MP4 file formats for podcast recording, editing, and publishing.

6.3 Hardware interfaces

HI-1*:* The system shall allow access to a device's microphone for recording podcasts.

6.4 Communications interfaces

CI-1:The application shall provide notifications to creators and viewers about audience interactions, and direct messages, ensuring seamless engagement.
CI-2:The system shall enable instant messaging and calls between users using real-time communication protocols, ensuring low-latency interactions.

7. References

List any documents or other resources to which this SRS refers, if any. These might include user interface style guides, standards, system requirements specifications, interface specifications, or the SRS for a related product. The following are a few examples of different resources.

Book
Author(s). Book title. Location: Publishing company, year, pp.
Example:
W.K. Chen. Linear Networks and Systems. Belmont, CA: Wadsworth, 1993, pp. 123-35.

Article in a Journal
Author(s). "Article title". Journal title, vol., pp, date.
Example:
G. Pevere. "Infrared Nation." The International Journal of Infrared Design, vol. 33, pp. 56-99, Jan. 1979.

Articles from Conference Proceedings (published)
Author(s). "Article title." Conference proceedings, year, pp.
Example:
D.B. Payne and H.G. Gunhold. "Digital sundials and broadband technology," in Proc. IOOC-ECOC, 1986, pp. 557-998.

World Wide Web
Author(s)*. "Title." Internet: complete URL, date updated* [date accessed].
M. Duncan. "Engineering Concepts on Ice. Internet: www.iceengg.edu/staff.html, Oct. 25, 2000 [Nov. 29, 2003].